



Fundamentos de IA e LLMs para Programadores

Autoria: Erick Wendel

Introdução

Ao longo das aulas, explorei os principais fundamentos por trás de sistemas de IA modernos, especialmente aqueles que utilizam o conceito de RAG — *Retrieval-Augmented Generation*. Agora, chegou o momento de transformar teoria em prática.

Neste capítulo, apresento a implementação do meu primeiro projeto funcional de RAG utilizando apenas JavaScript e Neo4j. Minha intenção foi demonstrar que é possível construir uma arquitetura robusta de recuperação e consulta de informações sem depender de soluções prontas ou serviços de terceiros. Mais do que isso, quis mostrar que qualquer programador com familiaridade em JavaScript pode aplicar esse conhecimento em cenários reais.

Decidi focar em uma abordagem local, leve e acessível, para deixar claro como os componentes do RAG — como embeddings, vetores de similaridade e bancos vetoriais — se integram na prática. A ideia é sair do discurso genérico e colocar a mão no código, entendendo cada etapa do processo com clareza e propósito.

Ao final deste capítulo, você terá uma visão sólida de como estruturar seu próprio sistema de recuperação de contexto, abrindo caminho para soluções personalizadas, escaláveis e com controle total sobre os dados utilizados na construção de aplicações baseadas em LLMs.

Módulo 01: Introdução ao curso

Capítulo 1: Introdução ao curso

Seja muito bem-vindo à pós-graduação em Engenharia de Software com Inteligência Artificial Aplicada. Eu sou Erick Wendel e estou aqui para te acompanhar nessa jornada de aprendizado, focada em algo que vai muito além de teoria: aplicação prática e transformação de conhecimento em resultado.

Antes de mergulharmos profundamente nos tópicos mais complexos, quero compartilhar com você uma visão geral sobre o que esperar do curso, como tirar o máximo proveito do conteúdo, e, principalmente, como transformar cada aula em uma oportunidade real de crescimento profissional.

Meu nome é Erick Wendel, e talvez você já tenha me visto sendo reconhecido por empresas como Google, Microsoft, GitHub e OpenJS Foundation, especialmente pela minha atuação com desenvolvimento web. Com mais de uma década de experiência em desenvolvimento de software, sempre busquei ir na contramão do hype, focando no que realmente funciona e no que traz valor ao desenvolvedor.

Embora meu foco recente tenha sido JavaScript, meu histórico envolve diversas linguagens, tecnologias e conceitos. Isso me permite trazer uma visão ampla sobre o ecossistema de desenvolvimento, e agora, com o foco em inteligência artificial, quero te mostrar como esses conhecimentos convergem de forma poderosa.

Logo de início, quero te fazer um pedido importante: foque em prática, questione, participe e aplique. Muitos alunos se matriculam, assistem as primeiras aulas e depois deixam o curso de lado. O resultado disso é previsível: pouco aprendizado, sensação de tempo e dinheiro desperdiçados. Não quero que você seja mais um nesse grupo.

Quero que você realmente aprenda. E, para mim, aprender vai muito além de assistir às aulas. Aprender significa pegar os exemplos ensinados, adaptá-los a contextos diferentes, criar projetos no GitHub com base nos conceitos explorados em aula e compartilhar esse conhecimento com a comunidade.

Se em uma aula eu te mostrar como processar imagens com uma abordagem de IA, desafio você a criar um projeto que processe áudios ou vídeos. Se o exemplo envolver boas práticas para lidar com dados em PDF, proponho que você faça o mesmo com arquivos CSV, planilhas ou outros formatos. A ideia é expandir, explorar e testar os limites do que você aprende aqui.

Outro ponto fundamental é a leitura complementar. Sempre que eu sugerir artigos, documentações ou materiais extras, leia com atenção, coloque em prática e leve suas dúvidas para o nosso canal no Discord. Essa interação é essencial para consolidar o aprendizado e fomentar discussões ricas entre os colegas da pós.

O aprendizado real acontece fora do tempo de aula. É quando você abre o seu editor de código, encara um problema real, tenta aplicar um conceito e percebe que surgem dúvidas, ideias e novos caminhos. Esse é o momento em que o conhecimento se transforma em competência.

Por isso, recomendo fortemente que você desenvolva um projeto pessoal ao longo do curso. Pode ser algo pequeno no início, mas com potencial de evoluir. Compartilhe o andamento na comunidade. Além de motivar outros alunos, você se coloca em evidência, gerando oportunidades de networking, parcerias e até mesmo indicações profissionais.

Nossa comunidade no Discord está ativa e é um espaço seguro para trocar ideias, compartilhar experiências, tirar dúvidas e, principalmente, mostrar seus avanços. Sugiro que você mantenha o aplicativo ou uma aba do navegador aberta com a comunidade ativa. Participe, mesmo que com pequenas contribuições. Isso faz toda a diferença.

Agora, um ponto crucial sobre o foco desta formação: aqui nós não estamos formando cientistas de dados. Esta pós-graduação é voltada para desenvolvedores. Quando falamos de inteligência artificial, é comum encontrar conteúdos que mergulham profundamente em matemática, estatística e infraestruturas complexas. Nosso objetivo aqui é diferente.

Queremos que você aprenda a usar IA no seu dia a dia de desenvolvimento. Isso inclui entender os conceitos, conhecer boas práticas, explorar ferramentas e plataformas acessíveis, sejam elas gratuitas ou pagas, e aplicar soluções de IA diretamente no navegador ou em projetos locais.

Com isso, você não só amplia seu leque de competências, mas se torna capaz de resolver problemas reais com ferramentas modernas, usando IA como um

diferencial competitivo. E mais: com foco em projetos práticos, você constrói um portfólio que demonstra sua capacidade de aplicação, não apenas de compreensão teórica.

Reforço a importância de você avaliar cada aula. Deixe seus comentários, reflexões e perguntas. Essa é a forma que eu tenho de entender se o conteúdo está fazendo sentido pra você, se estamos no caminho certo e, principalmente, de ajustar a abordagem conforme a necessidade da turma.

Avaliar, comentar e interagir é parte do processo de aprendizado. Suas dúvidas podem ser as mesmas de outros alunos, e ao compartilhá-las, você não apenas se ajuda, mas ajuda toda a comunidade.

Por fim, deixo aqui meu compromisso com você: vou entregar o melhor conteúdo possível, com base em experiências reais, no que o mercado precisa e no que você pode aplicar desde já. Mas preciso do seu comprometimento também. Não consuma esse curso como um conteúdo passivo. Coloque a mão na massa, compartilhe, participe, questione.

Estou aqui pra te ajudar. Conte comigo nessa jornada. Vamos juntos tornar a inteligência artificial uma ferramenta presente no seu dia a dia de desenvolvedor.

Bora começar!

Capítulo 2: O que você verá durante o curso, como praticar e nossa comunidade

Este primeiro módulo do curso marca sua entrada definitiva no universo da inteligência artificial aplicada, com foco total em resultados práticos. A proposta aqui é clara: sair do discurso mágico sobre IA e entrar em um cenário onde você realmente entende o que está fazendo, como funciona e, principalmente, como aplicar no seu dia a dia de trabalho.

Logo de início, você vai ter contato com conceitos fundamentais, mas sempre com exemplos reais e utilidade direta. Vamos abordar tópicos como JavaScript,

WebAI, LLMs, Reg, Embeddings, Vector Databases e Agentes de IA. Você também vai conhecer e utilizar ferramentas modernas como Teachable Machine, Oriama, OpenRouter e o próprio VSCode adaptado para produtividade com IA.

A jornada começa diferenciando conceitos que muitas vezes se confundem: Machine Learning, Deep Learning e Inteligência Artificial. Em seguida, mergulhamos no funcionamento das redes neurais, exploramos sistemas de recomendação, compreendemos o papel de estruturas como Transformers, Attention e Embeddings dentro dos LLMs. Mas não paramos na teoria. Cada conceito é aplicado na prática.

Ao longo do módulo, você irá entender como aplicar essas técnicas na web, utilizando IA para criar assistentes virtuais, otimizar processos e construir soluções inovadoras. Também iremos explorar como a IA toma decisões, como otimizar algoritmos e aplicar estratégias de resolução de problemas. Vamos discutir também algoritmos genéticos, um tema que ajuda a entender a evolução de soluções e que pode ser aproveitado em diversos contextos.

Na parte de LLMs, o foco é total em prompts eficazes, padrões como Toon e JSON, e formas de tornar suas interações com modelos grandes de linguagem escaláveis, reproduzíveis e testáveis. Você entenderá, por exemplo, como transformar uma ideia em uma instrução precisa que gera o resultado esperado com consistência.

Do ponto de vista das ferramentas, vamos comparar ambientes como VSCode, Cursor e Windsurf, destacando como cada um influencia diretamente na produtividade. Essa parte é essencial porque o uso correto das ferramentas certas é o que permite que você implemente IA com agilidade e robustez.

Outro ponto de destaque do módulo é o estudo sobre agentes de IA e como eles funcionam por etapas, executando processos completos com base em decisões automatizadas. Aí você entra em contato com o MCP (Model Context Protocol), uma ponte entre modelos de IA e APIs externas, permitindo automações muito mais complexas do que apenas interações via chat.

Com essa base, o curso se torna uma verdadeira caixa de ferramentas. Você vai aprender a utilizar IA para automatizar testes, navegar em sites, consultar documentações, extrair dados, e até mesmo interpretar dados de telemetria de aplicações. Cada aula trará uma aplicação prática.

O módulo também entra na discussão entre modelos de LLMs abertos e fechados, apresentando vantagens e desvantagens, bem como os caminhos para executar modelos localmente usando Yama. Além disso, você aprenderá a orquestrar diferentes modelos com OpenRouter, balanceando custo, desempenho e privacidade de forma inteligente, assim como fazem os times mais modernos do mercado.

Uma das partes mais importantes é o aprendizado sobre Reg (Retrieval Mentor Generation), um conceito essencial para sistemas de IA em produção. Você aprenderá a criar seu primeiro Reg usando JavaScript e Neo4j, integrando Embeddings e bancos de dados vetoriais. Isso te prepara para desenvolver assistentes baseados em dados reais do seu negócio.

Vale destacar que, embora este seja o primeiro módulo, ele não é superficial. Ao contrário: você vai colocar a mão na massa desde a primeira aula. Serão mais de 12 projetos práticos ao longo do módulo, todos com potencial de uso real em empresas, produtos e projetos pessoais ou de clientes. Não é apenas sobre entender IA, mas sim sobre saber como aplicá-la de forma concreta.

E por que usar JavaScript como linguagem principal? Embora a IA seja tradicionalmente associada ao Python, o JavaScript tem uma vantagem única: é a linguagem reconhecida pelos navegadores. Isso significa que qualquer desenvolvedor, mesmo que venha de outra linguagem, vai inevitavelmente ter que interagir com JavaScript em algum momento.

Se você é desenvolvedor web, tem em mãos poderes que poucos têm: a capacidade de rodar IA diretamente no navegador. Com a evolução da Web 4.0, os sites estão sendo adaptados para que inteligências artificiais possam buscar respostas direto na fonte. Mas o JavaScript não serve apenas pra isso. Ele permite que você utilize APIs nativas dos navegadores para executar modelos

de IA, processar áudio, vídeo, criar interfaces inteligentes e muito mais, tudo sem custo adicional e com performance otimizada.

Isso coloca você à frente de quem apenas integra APIs de terceiros. Aqui, você vai aprender a usar IA de forma eficiente, moderna e com foco na realidade do mercado. E o melhor: com economia de recursos e autonomia técnica.

Por fim, reforço a importância da nossa comunidade. Todo esse aprendizado ganha força quando é compartilhado. Ao participar ativamente do nosso canal no Discord, você ganha acesso a discussões relevantes, troca de experiências, feedbacks e oportunidades reais de crescimento profissional. Compartilhe suas dúvidas, mostre seus avanços e contribua com o aprendizado coletivo.

Estamos apenas começando. Espero que você esteja tão empolgado quanto eu. O conteúdo está só começando, mas o impacto que ele pode ter na sua forma de desenvolver software pode ser revolucionário.

Vamos juntos para a próxima etapa!

Módulo 02: Machine Learning, Deep Learning e IA - a base de tudo

Capítulo 1: Machine Learning, Deep Learning e Inteligência Artificial

Vivemos um momento em que termos como Inteligência Artificial, Machine Learning e Deep Learning estão em alta. Seja em postagens do LinkedIn, em artigos ou em produtos no mercado, é comum se deparar com essas expressões sem entender exatamente o que significam. Nesta aula, meu objetivo é desmistificar esses conceitos, mostrar que são mais acessíveis do que parecem e explicar como se aplicam, especialmente no contexto do desenvolvimento web.

O que é Inteligência Artificial?

O termo Inteligência Artificial (IA) é usado para descrever algoritmos que conseguem executar tarefas que, em teoria, exigiriam alguma forma de inteligência. Ao contrário do que o nome sugere, IA não tenta replicar a inteligência humana em si. Ela usa nomenclaturas inspiradas no funcionamento do cérebro humano, como "neurônios" e "redes neurais", mas os mecanismos por trás são essencialmente matemáticos e estatísticos.

De forma simples, podemos dizer que IA é qualquer sistema que aprenda com dados para executar uma tarefa. Quando falamos de sistemas que aprendem com dados, estamos nos referindo diretamente ao campo do Machine Learning.

Machine Learning: o coração da IA moderna

Machine Learning (ML), ou Aprendizado de Máquina, é uma subárea da IA focada em algoritmos que identificam padrões e fazem previsões a partir de dados. ML é usado em uma infinidade de aplicações, desde sistemas de recomendação até assistentes virtuais.

Na prática, ML substitui a necessidade de escrever regras manuais por modelos que aprendem com exemplos. Isso traz maior flexibilidade, escalabilidade e adaptabilidade aos sistemas.

Deep Learning: uma especialização poderosa

Deep Learning é uma especialização dentro do Machine Learning que utiliza redes neurais profundas para realizar tarefas mais complexas, como reconhecimento de imagem, voz, ou mesmo movimentos. A ideia central é que, ao empilhar várias camadas de "neurônios artificiais", a rede se torna capaz de aprender padrões cada vez mais abstratos.

Na minha experiência prática, isso ficou claro em um projeto que tentei desenvolver cerca de 10 anos atrás. Eu queria usar um smartwatch para controlar minhas apresentações, detectando gestos com o braço. Desenvolvi uma solução em Android puro, usando Java, para capturar os dados dos sensores (acelerômetro, giroscópio, etc.) e interpretar movimentos.

Apesar de avançar bastante, enfrentei vários desafios. Precisava calibrar os sensores manualmente, ajustar regras fixas para detectar movimentos e lidava com falsos positivos constantemente. Esse tipo de solução baseada em regras é limitada e pouco robusta.

Com Deep Learning, esse problema seria resolvido em minutos. Em vez de tentar escrever regras, basta coletar exemplos reais de movimentos, rotulá-los e treinar o modelo. O sistema aprende padrões temporais, reduzindo a necessidade de ajuste fino.

A aplicação prática no navegador

Para demonstrar isso de forma prática, usei uma biblioteca chamada TensorFlow.js, do Google. Com ela, é possível rodar modelos de Machine Learning e Deep Learning diretamente no navegador, sem back-end. Uma das demonstrações mais interessantes é um jogo de Pac-Man controlado com o movimento da cabeça.

Na demonstração, você define os comandos do jogo (cima, baixo, esquerda, direita) com movimentos faciais. O modelo é treinado com poucos exemplos e, em tempo real, é capaz de interpretar os gestos e controlar o jogo. Tudo isso executado no navegador.

Esse processo se baseia em uma técnica chamada transfer learning (ou transferência de aprendizado), em que usamos um modelo já treinado para reconhecer rostos e refinamos esse conhecimento com novos dados (os movimentos que você define).

E o que tem por trás do ChatGPT?

Um ponto de curiosidade comum é entender como modelos como o ChatGPT funcionam. De forma resumida, eles combinam diversos modelos de ML especializados. Ao interpretar sua pergunta, o sistema entende sua intenção e direciona o processamento para o modelo mais adequado. Esse processo ocorre de forma transparente, e o resultado é a resposta contextualizada.

O que vem pela frente

Durante este módulo, você vai aprender os fundamentos do ML, DL e IA com foco em aplicação web. Vamos falar sobre tensores, redes neurais, como criar e treinar modelos do zero, e como aplicar tudo isso nos seus projetos.

A tecnologia já está madura, acessível e pronta para ser usada. E você vai aprender a usar exatamente a mesma biblioteca do exemplo do Pac-Man: o TensorFlow.js. Essa biblioteca não apenas permite a criação de modelos do zero, como também possibilita portar modelos treinados em Python para execução em navegadores ou no Node.js.

Ela é amplamente usada em aplicações de visão computacional, como reconhecimento facial, leitura de letras escritas à mão, identificação de objetos, entre outros. E é com ela que você vai trabalhar bastante nesse módulo.

Por fim, o mais importante: enquanto estuda, pense onde e como você pode aplicar esses conhecimentos. Essa é a melhor forma de fixar o aprendizado e criar projetos próprios com base nos conceitos aprendidos.

Vamos em frente!

Capítulo 2: Teachable Machine

Nesta aula, apresento o Teachable Machine, uma ferramenta gratuita do Google que permite treinar modelos de Machine Learning de forma extremamente intuitiva, sem escrever uma linha de código. Com ele, é possível usar dados de imagem, áudio ou webcam para identificar padrões e classificar objetos, gestos ou sons.

Começamos explorando a interface da ferramenta, que suporta frameworks como Arduino, Node.js, P5.js e, claro, TensorFlow.js. A proposta inicial é mostrar que qualquer pessoa pode treinar um modelo simples, exportá-lo em HTML e integrá-lo em um site.

Criando seu primeiro modelo com a webcam

A primeira demonstração prática foi baseada em imagens capturadas pela webcam. Treinamos o modelo com três objetos físicos: uma garrafa, um controle remoto e um relógio. Para cada item, criei uma classe, capturei várias imagens em diferentes ângulos e iluminações, e usei a função de gravação da webcam para coletar exemplos visuais.

O modelo foi treinado diretamente no navegador e, após o treinamento, testamos a precisão da classificação. Os resultados foram surpreendentes: o modelo identificou corretamente todos os objetos, inclusive quando rotacionados ou parcialmente fora do centro da imagem.

Esse processo demonstra o conceito de reconhecimento de padrões visuais, mostrando que, mesmo sem backend ou infraestrutura complexa, é possível criar modelos funcionais com recursos simples.

Exportando e usando o modelo

Após o treinamento, é possível exportar o modelo como um arquivo HTML. Esse arquivo inclui **todo o código** necessário para executar o modelo no navegador, incluindo o uso da webcam para novas predições em tempo real. Você pode usar esse modelo em projetos práticos, como detectar objetos específicos e disparar ações automatizadas, como abrir uma porta ou acionar um alarme.

Aplicando com bases reais do Kaggle

Na segunda parte da aula, trouxemos um exemplo mais próximo da realidade de projetos profissionais. Utilizei uma base de dados pública do Kaggle contendo quase 8 mil imagens de diferentes raças de cachorros, com divisão entre dados de treinamento, validação e teste.

O processo envolveu o upload das imagens organizadas em pastas (uma para cada raça) no Teachable Machine. Treinamos o modelo com diversas classes, e em seguida testamos com imagens do conjunto de teste, que nunca haviam sido vistas pelo modelo. O desempenho foi preciso: o modelo foi capaz de identificar corretamente as raças conhecidas e, nos casos de imagens fora do treinamento, indicava a classe mais próxima com base na similaridade.

Esse experimento destaca dois conceitos-chave:

- Generalização: o modelo consegue fazer previsões precisas com dados que nunca viu, desde que os exemplos de treinamento sejam representativos.
- Transfer learning: mesmo com uma pequena quantidade de dados, é possível aproveitar modelos pré-existentes para refinar o aprendizado, como vimos com o uso do modelo base para reconhecimento facial.

Possibilidades e aplicações práticas

As aplicações vão muito além de jogos ou experimentos pessoais. Com um modelo treinado, é possível, por exemplo, identificar quando um animal de estimação está presente em determinado local e acionar um alimentador automático. Também é viável reconhecer pessoas específicas com base em gestos, voz ou expressões.

A chave é alimentar o modelo com muitos exemplos e garantir que as classes estejam bem representadas. Em projetos reais, é comum usar scripts para fazer upload massivo de dados e automatizar etapas do processo de treinamento.

Conclusão e desafio

Finalizei a aula com um desafio: explorar o Kaggle, encontrar uma base de dados de seu interesse e treinar um modelo personalizado usando o Teachable Machine. É um excelente exercício para entender como a IA aprende e como você pode adaptá-la para problemas reais.

O mais importante é entender que, com ferramentas acessíveis como essa, você já pode construir soluções baseadas em IA, testá-las no navegador e aplicar esses conhecimentos nos seus projetos profissionais.

Na próxima aula, voltaremos ao foco em conceitos para fortalecer ainda mais sua base teórica.

Seguimos!

Capítulo 3: Conceito de redes neurais e como elas aprendem

Capítulo 9 - Conceito de Redes Neurais e Como Elas Aprendem

Nesta aula, nos aprofundamos nos fundamentos de como uma rede neural funciona, desde a transformação dos dados em tensores até o processo de treinamento, explicando detalhadamente o que acontece entre a entrada e a saída de um modelo de IA.

Começamos com uma revisão prática do TensorFlow.js, biblioteca base para nossas aplicações. Como o nome sugere, o TensorFlow trabalha com tensores, que nada mais são do que vetores ou listas compostas por números. Esses tensores são a representação numérica de dados que o algoritmo de aprendizado de máquina pode interpretar e processar.

Transformando objetos em tensores

Para exemplificar, usamos uma estrutura com pessoas, contendo informações como idade, cor preferida e localização. Cada pessoa pertence a uma categoria (Premium, Medium ou Basic). O desafio é prever a categoria de uma nova pessoa com base nas suas características.

Como o TensorFlow.js não entende objetos do JavaScript diretamente, precisamos converter essas informações em estruturas puramente numéricas: os tensores. Aplicamos a técnica de categorização, transformando propriedades como cor e localização em one-hot encoding. Por exemplo, se há três cores distintas, cada pessoa recebe um vetor com uma única posição com valor 1 correspondente à sua cor preferida.

A idade, por ser um valor numérico contínuo, é normalizada para o intervalo entre 0 e 1, ajustando-se às escalas dos outros atributos. Isso evita que a idade tenha peso desproporcional no treinamento.

Normalização e one-hot encoding

Chamamos esse processo de normalização. Se temos pessoas com idades de 15 a 45 anos, normalizamos esses valores para que 15 vire 0, 45 vire 1 e os valores intermediários fiquem proporcionalmente distribuídos entre 0 e 1. Para cor e localização, usamos one-hot encoding: apenas uma posição do vetor é ativada com 1, as demais ficam com 0.

Essa estrutura gera um tensor bidimensional, onde cada linha representa uma pessoa. Esse formato padronizado é essencial para alimentar a rede neural com consistência.

Entendendo as redes neurais

Uma rede neural é formada por camadas: entrada, camadas ocultas e saída. Cada entrada (por exemplo, idade, cor e localização) é conectada a vários neurônios da primeira camada oculta. Cada neurônio funciona como uma "mini calculadora", que combina os valores recebidos, aplica fórmulas matemáticas (funções de ativação) e retorna um novo valor.

Esse valor alimenta a próxima camada, e assim por diante. Cada camada funciona como um filtro que extrai padrões mais abstratos. As conexões entre os neurônios são ajustadas durante o treinamento por meio de pesos, que determinam a influência de cada entrada.

Na camada de saída, cada neurônio representa uma categoria possível (Premium, Medium, Basic). A rede retorna uma probabilidade para cada categoria, permitindo identificar com qual grupo o novo dado se parece mais.

O que significa treinar uma rede neural?

Treinar é o processo de mostrar vários exemplos rotulados (como fizemos com Eric, Ana e Carlos), para que o algoritmo ajuste os pesos das conexões entre os neurônios. Quanto mais exemplos e mais variados, melhor o aprendizado.

Durante o treinamento, o algoritmo compara a saída gerada com a saída esperada (o rótulo correto) e ajusta os pesos para reduzir o erro. Esse processo se repete por várias iterações até que a rede atinja um nível aceitável de precisão.

Importância da quantidade e diversidade dos dados

Para que a rede seja robusta, ela precisa ver exemplos variados. Isso foi demonstrado em aplicações reais como o sensor LEM do Google, que usou mais de 60 milhões de horas de dados para treinar um modelo que detecta variações de batimento cardíaco e temperatura.

Por isso, para problemas reais, quanto mais dados e mais diversidade, melhor. Nós, desenvolvedores web, podemos usar modelos prontos ou personalizá-los, e é justamente isso que vamos explorar ao longo do curso.

Conclusão

Nesta aula, você entendeu como funciona uma rede neural, desde a entrada de dados até a saída preditiva, passando pelas transformações necessárias para representar os dados como tensores. Com esse conhecimento, você está pronto para construir suas próprias redes neurais, entender seu comportamento e aplicar em casos práticos.

Na próxima aula, vamos colocar isso tudo em prática, criando e treinando uma rede neural simples com base em tudo que foi discutido aqui.

Vamos seguir!

Capítulo 4: Criando e Treinando minha primeira Rede Neural para determinar a categoria de alunos PT 01

Chegou o momento de colocar em prática os conceitos discutidos nas aulas anteriores. Nesta aula, mostro passo a passo como criar e treinar uma rede neural simples usando TensorFlow.js no Node.js. A ideia central é aplicar, de forma concreta, os conhecimentos de tensores, normalização, categorização e arquitetura de rede neural para resolver um problema real: prever a categoria de um aluno com base em suas características.

Preparando o ambiente

Inicialmente, reforço que estou utilizando o Node.js na versão 22 para evitar problemas de compatibilidade com bibliotecas. É essencial que você utilize essa mesma versão para garantir que tudo funcione corretamente.

Reuni todas as entradas e saídas que discutimos anteriormente em um arquivo já preparado. Ele contém os dados de três pessoas: Eric, Ana e Carlos, com informações como idade, cor preferida e localização. Essas informações foram convertidas manualmente em tensores usando as mesmas técnicas vistas anteriormente: normalização para a idade e one-hot encoding para as categorias de cor e localização.

Estrutura dos dados

A estrutura final dos dados possui sete atributos: um para idade (normalizada) e seis para categorização (três cores e três localizações). Cada linha representa uma pessoa, e cada pessoa pertence a uma categoria (Premium, Medium ou Basic), representadas também por one-hot encoding.

Arquitetura da rede neural

Criei uma rede neural sequencial com duas camadas principais:

1. Camada Oculta:
 - Input shape: 7 (um para cada atributo).
 - 80 neurônios: escolhidos empiricamente para aumentar a capacidade de aprendizado mesmo com poucos dados.
 - Função de ativação: *ReLU* (Rectified Linear Unit), que permite que apenas valores positivos passem adiante.
2. Camada de Saída:
 - 3 neurônios: um para cada categoria.
 - Função de ativação: *Softmax*, que transforma os valores de saída em probabilidades normalizadas.

Essa arquitetura simples é suficiente para demonstrar os conceitos fundamentais do treinamento de modelos.

Compilando o modelo

Utilizei o otimizador Adam (Adaptive Moment Estimation), que ajusta os pesos da rede com base no histórico de erros e acertos durante o treinamento. Para a função de perda (*loss function*), optei por Categorical Cross Entropy, amplamente usada em problemas de classificação.

Também defini a métrica de desempenho como a própria *loss*, o que nos permite avaliar o quão distante a saída do modelo está da resposta correta.

Treinamento do modelo

A etapa de treinamento foi feita com as seguintes configurações:

- Epochs: 100 (o modelo percorre 100 vezes os dados de entrada).
- Shuffle: true (os dados são embaralhados a cada iteração para evitar viés).
- Callback de log: registra o valor da *loss* a cada epoch, permitindo acompanhar a evolução do aprendizado.

Durante o treinamento, a *loss* começou alta, como esperado, e foi diminuindo conforme o modelo ajustava os pesos. Em poucas iterações, a rede já demonstrava sinais claros de aprendizado, reduzindo consideravelmente o erro.

Considerações finais

Mesmo com uma base de dados pequena, é possível observar o funcionamento completo de uma rede neural: da entrada ao treinamento e saída. À medida que mais dados forem adicionados, a capacidade de generalização do modelo aumenta significativamente.

Este exemplo mostra de forma concreta o poder do TensorFlow.js e como você, desenvolvedor, pode implementar IA nos seus projetos sem precisar sair do ambiente JavaScript.

Na próxima aula, vamos explorar como testar este modelo e fazer previsões com novos dados.

Vamos em frente!

Capítulo 5: Criando e Treinando minha primeira Rede Neural para determinar a categoria de alunos PT 02

Dando continuidade à aula anterior, chegou o momento de colocarmos nosso modelo à prova. Com a rede neural já treinada, nossa missão agora é testar se ela realmente aprendeu a categorizar novos dados corretamente. Para isso, criamos um novo exemplo de aluno, que não fez parte do treinamento, e observamos como o modelo responde.

Criando um novo aluno para teste

Começamos criando um aluno fictício chamado Zé. A ideia era simular um caso semelhante ao de um aluno já conhecido, o Carlos, que pertence à categoria Basic. O novo aluno tem as seguintes características:

- Cor preferida: verde
- Localização: Curitiba
- Idade: 28

Essas informações foram escolhidas estrategicamente para verificar se a rede neural seria capaz de associar padrões semelhantes aos aprendidos durante o treinamento.

Normalizando os dados do novo aluno

Antes de alimentar o modelo, normalizamos os dados seguindo o mesmo padrão usado durante o treinamento:

- Idade mínima: 25 (referente à Ana)
- Idade máxima: 40 (referente ao Carlos)
- Idade normalizada: $(28 - 25) / (40 - 25) = 0.2$

A cor verde foi codificada com o valor 1 no vetor correspondente, assim como a localização Curitiba. O resultado foi um vetor de entrada com os mesmos sete elementos usados no treinamento.

Realizando a predição com o modelo treinado

Com os dados preparados, criamos uma função chamada `predict` que recebe o modelo treinado e o vetor do novo aluno. Usamos `tf.tensor2d` para transformar os dados em um tensor bidimensional, garantindo compatibilidade com o modelo.

A predição retorna um array com três probabilidades, uma para cada categoria: Premium, Medium e Basic. O valor mais alto indica a categoria com maior probabilidade segundo a rede.

Na primeira execução, os resultados foram:

- Premium: 8%
- Medium: 13%
- Basic: 77%

O modelo corretamente identificou o perfil do Zé como mais próximo da categoria Basic, mesma categoria de Carlos, demonstrando que a rede aprendeu padrões relevantes.

Explorando variações nos dados

Para testar a sensibilidade do modelo, modificamos algumas características do aluno e observamos o impacto na predição:

1. Mudança de localização para São Paulo:
 - A predição mudou significativamente, indicando 42% de probabilidade para a categoria Premium.
 - Isso ocorre porque Eric, categorizado como Premium, também é de São Paulo e tem idade semelhante.
2. Mudança da cor preferida para azul:
 - A probabilidade de Premium aumentou ainda mais, para 90%.
 - A rede neural reconheceu o padrão semelhante ao de Eric, reforçando a capacidade do modelo de associar características aprendidas.
3. Combinação de padrões não vistos:

- Ao testar padrões que não estavam presentes no treinamento, o modelo ainda apontava uma categoria, mas com confiança reduzida e distribuição mais equilibrada entre as opções.

Esses testes demonstram como o modelo se comporta com diferentes entradas e evidenciam a importância de uma base de dados rica e variada para melhorar a precisão da rede.

Visualizando os resultados

Utilizamos funções de ordenação e mapeamento para exibir os resultados de forma mais amigável, mostrando as categorias em ordem decrescente de probabilidade. Isso facilita a interpretação do comportamento da rede e permite ajustes mais precisos no treinamento.

Considerações finais

Com esta aula, encerramos o ciclo de criação, treinamento e validação de uma rede neural simples. Você aprendeu como:

- Normalizar novos dados de entrada
- Reutilizar um modelo treinado para novas previsões
- Interpretar as saídas de uma rede neural
- Ajustar entradas para verificar a sensibilidade do modelo

Esse é um passo fundamental para o desenvolvimento de sistemas inteligentes. Mesmo com uma base pequena, já foi possível visualizar o potencial de redes neurais na categorização de dados e na tomada de decisões baseadas em padrões aprendidos.

Na próxima aula, vamos elevar ainda mais o nível, criando um sistema de recomendação no navegador utilizando os mesmos conceitos.

Vamos em frente!

Módulo 03: Deep Learning - Sistemas de Recomendação na prática

Capítulo 1: Como funcionam Sistemas de recomendação PT01

Neste capítulo, começamos a explorar o funcionamento de sistemas de recomendação aplicados diretamente no navegador, utilizando TensorFlow.js. Diferente dos exemplos anteriores, agora vamos trabalhar com um volume maior de dados e aplicar os conhecimentos adquiridos para construir um modelo que sugere produtos personalizados com base em histórico de compras de usuários.

O desafio da recomendação personalizada

Nosso objetivo é ordenar uma lista de produtos com base na probabilidade de um determinado cliente comprá-los. Se for um cliente novo, o sistema deve considerar os atributos de outros compradores com perfil semelhante. Isso exige um modelo treinado com uma base rica e variada, pois quanto mais exemplos forem fornecidos, maior a capacidade de previsão.

Estrutura do projeto

Para facilitar, disponibilizei um template com todo o layout pronto, incluindo a interface de monitoramento do treinamento com tfvis, uma biblioteca do TensorFlow.js. O projeto é executado inteiramente no navegador, com suporte a multithreading através de *Web Workers*, o que permite realizar o treinamento de redes neurais em segundo plano, sem travar a interface.

A estrutura de dados utilizada inclui:

- Nome do usuário
- Idade
- Lista de compras
- Categoria do produto
- Preço

- Cor

Essas informações serão usadas para treinar o modelo e estabelecer relações entre o perfil do cliente e os produtos adquiridos.

Simulação com dados reais

Incluimos dados fictícios de vários usuários, como Ana Lima e Bruno, com suas respectivas idades e histórico de compras. O desafio é identificar padrões de comportamento que permitam prever quais produtos devem ser recomendados para um novo usuário, como o Zezinho da Silva, que não participa do treinamento.

O sistema tenta encontrar similaridades de perfil para sugerir os produtos mais prováveis de serem comprados. Isso pode se basear em atributos como idade, categoria de produtos preferida, ou mesmo o comportamento de compra de usuários semelhantes.

Interatividade e visualização do treinamento

Toda a interação é baseada em eventos. Ao adicionar ou remover compras para um usuário, o modelo é automaticamente atualizado. A cada treinamento, os gráficos do tfvis mostram em tempo real a evolução do modelo, permitindo visualizar métricas como perda e acurácia.

Essa abordagem baseada em eventos facilita o entendimento da dinâmica do sistema e permite testar diferentes combinações de dados para avaliar o impacto nas recomendações.

Implementação modular e foco em aprendizado

Toda a lógica da rede neural está isolada na pasta `workers`, o que permite focar exclusivamente na criação e treinamento do modelo. Importei previamente as dependências necessárias, como o TensorFlow.js, variáveis de controle de progresso e funções auxiliares para registro de logs e status do treinamento.

A proposta é que você implemente duas funções principais:

1. Criação da rede neural
2. Treinamento e atualização dos resultados

A interface visual e a estrutura de eventos já estão prontas, permitindo que você foque apenas na lógica de aprendizado, sem se preocupar com layout ou manipulação de DOM.

Recomendações baseadas em padrões

Mesmo que o cliente não tenha um histórico próprio, como no caso do Zezinho, o modelo pode utilizar a média dos comportamentos registrados para sugerir itens. Produtos populares, como um relógio inteligente comprado por vários usuários, podem ser recomendados com base em sua relevância geral no conjunto de dados.

Essa abordagem imita o funcionamento de sistemas de recomendação reais, como os utilizados por grandes plataformas de e-commerce e streaming.

Conclusão

Neste primeiro contato com sistemas de recomendação, construímos a base para um modelo funcional que roda 100% no navegador. Usamos dados simulados, mas com estrutura semelhante às bases reais, e aproveitamos recursos de multithreading para treinar redes neurais de forma eficiente.

Na próxima aula, vamos iniciar a implementação do modelo em si, ajustando os dados de entrada, criando a rede neural e realizando os primeiros testes de recomendação com base em dados reais.

Vamos em frente!

Capítulo 2: Como funcionam Sistemas de recomendação PT02

Neste segundo capítulo sobre sistemas de recomendação, damos continuidade à estruturação do modelo no navegador. Aqui, o foco é montar o contexto de treinamento com base no catálogo de produtos e dados dos usuários, transformando essas informações em entradas numéricas que o TensorFlow.js possa utilizar de forma eficiente. Essa é a base para que o modelo compreenda o comportamento dos usuários e possa fazer sugestões personalizadas.

Personalização por pesos e categorias

Iniciamos com a ideia de que, em sistemas de recomendação, não basta apenas saber o que o usuário comprou, mas sim atribuir pesos diferentes a essas compras. Por exemplo, se um usuário comprou uma camiseta, seria interessante priorizar a recomendação de camisetas semelhantes. O mesmo se aplica a outras categorias como calçados ou eletrônicos. Essa abordagem torna o sistema mais inteligente e customizado.

Estrutura do Worker e manuseio de dados

Todo o processamento acontece dentro de um *Web Worker*, separado da interface principal para garantir uma experiência fluida no navegador. O worker é iniciado via `new Worker` e escuta mensagens da interface. Cada mensagem representa um evento, como iniciar o treinamento ou executar uma predição.

Carregamos o catálogo de produtos da pasta `data`, acessando um arquivo JSON que contém informações como nome, preço, categoria e cor. Esses dados são a base para estruturar o contexto de treinamento.

Montando o contexto

A construção do contexto começa pela coleta de informações relevantes:

- Idades dos usuários
- Preços dos produtos
- Cores e categorias presentes no catálogo

Usamos funções como `Math.min` e `Math.max` para calcular os intervalos de idade e preço, que depois serão usados para normalização. Criamos também tabelas

de índice para cores e categorias, usando `Object.entries` e `Set` para evitar repetições.

Cálculo da média de idade por produto

Uma etapa importante para a personalização é calcular a média de idade dos compradores de cada produto. Isso permite que o modelo compreenda que certos itens são mais populares entre faixas etárias específicas. Por exemplo, se um fone de ouvido é mais comprado por pessoas entre 25 e 30 anos, o sistema pode usar essa informação para ajustar a recomendação conforme a idade do novo usuário.

Criamos dois objetos auxiliares: `ageSums` para somar as idades por produto, e `ageCounts` para contar quantas vezes cada produto foi comprado. Com isso, calculamos a média real de idade por produto e, se não houver dados suficientes, usamos a média geral como fallback.

Normalização dos dados

Utilizamos a mesma lógica de normalização vista em aulas anteriores, transformando valores de idade e preço para o intervalo entre 0 e 1. Isso garante que todas as variáveis tenham peso proporcional no treinamento da rede.

A função de normalização segue a fórmula padrão:

$$(\text{valor} - \text{min}) / (\text{max} - \text{min})$$

Se o resultado for indefinido, retornamos 1 como valor padrão.

Resultado final do contexto

Com todas as informações processadas, estruturamos o contexto com:

- Catálogo de produtos
- Usuários
- Índice de cores e categorias
- Idade e preço mínimo e máximo
- Número total de categorias e cores

- Dimensão total da entrada (idade + preço + categorias + cores)

Esse contexto será usado nas próximas etapas para transformar os dados em tensores e alimentar o modelo de recomendação.

Conclusão

Encerramos esta etapa com a base completamente estruturada para normalização e criação do contexto. Essa preparação é essencial para garantir que a rede neural compreenda corretamente os dados fornecidos e consiga realizar recomendações eficazes.

Na próxima aula, vamos utilizar esse contexto para transformar os dados em tensores e iniciar de fato o treinamento da nossa rede neural no navegador.

Vamos em frente!

Capítulo 3: Como funcionam Sistemas de recomendação PT03

Neste capítulo, damos continuidade à implementação do nosso sistema de recomendação no navegador. Depois de montar todo o contexto com os produtos e usuários, agora transformamos esses dados em tensores, etapa fundamental para alimentar a rede neural com informações estruturadas. O foco está em normalizar e aplicar pesos às variáveis para representar corretamente os produtos em vetores numéricos.

Codificando os produtos em tensores

O primeiro passo foi criar uma função chamada `encodeProduct`, responsável por transformar cada produto do catálogo em um vetor de atributos normalizados. Os atributos considerados foram:

- Preço
- Idade média dos compradores

- Categoria (one-hot encoding)
- Cor (one-hot encoding)

Definimos pesos para cada atributo, refletindo sua relevância na tomada de decisão da rede:

- Categoria: peso mais alto (prioridade na recomendação)
- Cor
- Preço
- Idade: menor peso, mas ainda importante para identificar padrões de consumo por faixa etária

A aplicação desses pesos sobre os atributos normalizados aumenta sua influência no vetor final, o que contribui para que o modelo dê mais importância a determinadas características.

Normalização dos atributos

Usamos a fórmula padrão para normalização:

$$(\text{valor} - \text{min}) / (\text{max} - \text{min})$$

Essa fórmula foi aplicada ao preço e à idade média de cada produto. No caso de produtos sem dados de idade associados, utilizamos um valor padrão (0.5) para evitar inconsistências.

Para as categorias e cores, aplicamos a técnica de one-hot encoding com multiplicadores, transformando a presença de um item em um vetor binário e aplicando o peso correspondente. Isso garante que essas variáveis categóricas sejam representadas numericamente de forma adequada e proporcional à sua importância.

Compondo os vetores finais dos produtos

Os vetores finais de cada produto foram compostos com a função `tf.concat`, unindo os tensores de preço, idade, categoria e cor em um único vetor. Esse vetor representa o produto de forma completa e pronta para ser usada como entrada da rede neural.

Cada vetor foi armazenado junto com seus metadados (nome e atributos do produto), permitindo o uso posterior tanto no treinamento quanto na exibição dos resultados de predição.

Tratamento de erros e ajustes

Durante a codificação, corrigimos inconsistências como a construção incorreta dos índices de categorias e cores, substituindo `Object.entries` por `Object.fromEntries` para garantir que as estruturas estivessem no formato correto.

Esse tipo de ajuste é essencial para garantir que o modelo seja treinado com dados consistentes e evite erros na execução. Como os objetos do TensorFlow.js têm uma estrutura complexa (incluindo `shape`, `dtype`, etc.), é importante validar os vetores gerados antes de usá-los.

Otimização para ambientes com muitos dados

Apesar de o projeto estar sendo feito inteiramente no navegador com uma base reduzida, discutimos como adaptar esse processo para ambientes com grandes volumes de dados. A sugestão é utilizar bancos de dados vetoriais para armazenar os vetores de produtos, permitindo buscar rapidamente os itens mais semelhantes ao perfil de um novo usuário.

Esse tipo de banco permite, por exemplo, retornar os 200 produtos mais similares ao perfil de entrada, economizando memória e tempo de processamento. Isso é especialmente útil em projetos de produção com milhões de usuários e produtos.

Encerramento da etapa de codificação de produtos

Com os produtos já convertidos em vetores e devidamente normalizados, completamos a etapa de estruturação das entradas da rede neural. O próximo passo será codificar os usuários, considerando suas compras, idade e padrões de comportamento, para então dar início ao treinamento do modelo de recomendação.

Estamos construindo o sistema do zero, compreendendo cada etapa da representação de dados e como ela impacta na qualidade das predições. Isso proporciona uma base sólida para entender como funcionam os sistemas de IA em projetos reais.

Na próxima aula, entraremos na etapa de codificação dos usuários.

Vamos em frente!

Capítulo 4: Como funcionam Sistemas de recomendação PT04

Neste capítulo, damos continuidade à construção do sistema de recomendação no navegador, focando agora na normalização e codificação dos usuários. Com os produtos já transformados em vetores, chegou a hora de fazer o mesmo para os usuários ativos na base, associando-os aos produtos comprados para montar os dados de treinamento da rede neural.

Criando os dados de treinamento

Comecei criando a função `createTrainingData`, que recebe o `context` como parâmetro. Dentro dessa função, percorremos cada usuário da base (`context.users`) e, para cada um deles, criamos seu vetor pessoal utilizando a função `encodeUser`.

A `encodeUser` funciona de maneira similar à `encodeProduct`: ela empilha os vetores dos produtos comprados pelo usuário usando `tf.stack`, depois calcula a média desses vetores para representar o perfil de compra daquele usuário. Isso nos permite construir um vetor numérico resumido para cada cliente.

Se o usuário não tiver compras, esse tratamento ainda não foi implementado, mas para os casos com compras, conseguimos gerar um vetor adequado com os dados normalizados e com pesos corretos.

Identificando se o usuário comprou determinado produto

Com os vetores de usuários prontos, passamos a verificar quais produtos cada um deles comprou. Para isso, percorremos o catálogo de produtos (renomeado de `catalog` para `products` por clareza) e usamos a função `encodeProduct` para gerar o vetor de cada produto.

Depois, comparamos o nome de cada produto com os itens da lista de compras do usuário. Se o produto foi comprado, atribuímos o rótulo `1`; caso contrário, `0`. Isso cria a *label* que será usada como saída esperada no treinamento da rede.

Combinando vetores de entrada

Para cada combinação de usuário e produto, juntamos seus vetores em um único vetor de entrada. Assim, o modelo saberá que aquele usuário, com determinado perfil, tem maior ou menor probabilidade de comprar aquele produto.

Esses vetores combinados são armazenados em um array `inputs`, enquanto as saídas esperadas (labels) vão para um array `labels`. Ao final, criamos os tensores `xs` e `ys` com `tf.tensor2d`, informando suas dimensões corretamente. O tamanho da entrada é calculado como `context.dimensions * 2`, pois estamos combinando dois vetores (usuário + produto).

Tratamento de erros e ajustes importantes

Durante o processo, identifiquei alguns erros e ajustes necessários:

- A variável global `context` estava sendo sobrescrita, então alterei sua declaração para `let`.
- Corrigi a utilização incorreta de `minPrice` em lugares onde o correto era `maxPrice`.
- Corrigi o uso de `numCategories` para cores, substituindo por `numColors` onde apropriado.
- Ajustei os pesos aplicados no `oneHotWeighted`, garantindo que categorias e cores estivessem sendo tratadas com os valores corretos.

Essas correções foram fundamentais para garantir que os vetores estivessem sendo montados com os dados corretos, sem gerar erros na execução.

Validação e inspeção dos dados

Utilizei `dataSync` para inspecionar os vetores gerados, conferindo se os valores estavam coerentes e se o TensorFlow estava interpretando corretamente as entradas. Essa etapa é importante para evitar problemas na fase de treinamento.

Com os dados processados, verificamos que os arrays `inputs` e `labels` estavam populados corretamente. Criamos os tensores de entrada (`xs`) e saída (`ys`) com as dimensões adequadas, prontos para serem usados na próxima etapa.

Conclusão

Concluimos com sucesso a etapa de criação dos dados de treinamento, combinando as informações dos usuários com os produtos e gerando os vetores que alimentam a rede neural. Essa parte, apesar de trabalhosa, é essencial para garantir que o modelo consiga aprender padrões reais e gerar recomendações eficazes.

Na próxima aula, finalmente iniciaremos o treinamento da rede neural e faremos as primeiras predições com base nesses dados estruturados.

Vamos em frente!

Capítulo 5: Como funcionam Sistemas de recomendação PT05

Neste capítulo, chegamos à etapa mais aguardada do projeto: o treinamento do modelo de recomendação no navegador. Com todos os dados codificados e normalizados, agora é o momento de configurar a rede neural, treinar com os dados e acompanhar a evolução em tempo real.

Configuração da rede neural

Criamos a função `configureNeuralNetAndTrain`, que recebe os dados de treinamento (`trainData`) e monta toda a estrutura da rede. Utilizamos o

`tf.sequential` para adicionar camadas de forma sequencial, assim como fizemos nas aulas anteriores.

A primeira camada recebe o `inputShape` com base na dimensão do vetor de entrada (`trainData.inputDimension`) e define 128 neurônios com função de ativação `relu`. Essa camada inicial é responsável por capturar os padrões iniciais dos dados.

Em seguida, adicionamos mais duas camadas densas com 64 e 32 neurônios, também usando `relu`, para refinar os padrões identificados. Essas camadas permitem que o modelo aprenda representações mais abstratas dos dados de entrada.

Finalizamos com uma camada de saída com apenas um neurônio e função de ativação `sigmoid`. Essa camada retorna uma probabilidade entre 0 e 1, indicando a chance do usuário comprar o produto.

Compilação do modelo

A compilação é feita com o otimizador `adam` e taxa de aprendizado de `0.01`, usando a função de perda `binaryCrossentropy` para problemas binários (compra ou não compra). Também adicionamos a métrica de `accuracy` para acompanhar o desempenho.

Treinamento do modelo

O treinamento é realizado com `model.fit`, utilizando os tensores `xs` e `ys` do `trainData`. Definimos 100 épocas, com `batchSize` de 32, `shuffle` ativado para evitar viés nos dados e um `callback` que registra os logs a cada época.

Durante o treinamento, os logs são enviados por `postMessage` com as informações de perda (`loss`) e acurácia (`accuracy`) de cada ciclo. Esses dados alimentam o painel de visualização `tfvis`, que mostra graficamente a evolução do modelo.

Tratamento de erros comuns

Durante os testes, corrigimos pequenos erros como o uso incorreto de `unit` (singular) em vez de `units` (plural), que causava falhas na inicialização da camada. Também adicionamos filtros para garantir que apenas usuários com compras fossem considerados no treinamento, evitando inconsistências.

Resultados visuais e interpretação

Após iniciar o treinamento, o gráfico do `tfvis` mostra a evolução do modelo. No início, a acurácia é baixa e a perda alta, mas com o avanço das épocas, o modelo se ajusta e alcança alta confiança nas predições. Por exemplo, valores de saída próximos de 0.9 indicam forte recomendação, enquanto valores próximos de 0.1 indicam baixa confiança.

Considerações finais

Com o treinamento funcionando e os resultados sendo exibidos, removemos o uso de `setTimeout` e utilizamos `await` para garantir que o modelo só envie mensagens após a finalização do treinamento.

Tivemos o cuidado de garantir que todos os elementos da interface estivessem sincronizados com o progresso do modelo, o que proporciona uma experiência interativa e didática.

Na próxima aula, vamos aplicar esse modelo para gerar recomendações reais para os usuários e encerrar o projeto com um ciclo completo de coleta, treinamento e predição.

Vamos em frente!

Capítulo 6: Como funcionam Sistemas de recomendação PT06

Neste último capítulo da série sobre sistemas de recomendação, chegamos à etapa final: realizar predições com o modelo treinado e exibir recomendações reais com base no perfil dos usuários, incluindo aqueles que não têm histórico

de compras. Isso fecha o ciclo completo do nosso sistema funcionando inteiramente no navegador.

Lidando com usuários sem histórico de compras

Para usuários que não possuem compras registradas, a função `encodeUser` foi modificada para gerar um vetor usando apenas a idade como entrada. Os demais atributos como preço, categoria e cor foram representados com tensores de zeros. A idade é normalizada usando a faixa `minAge` e `maxAge` definida no contexto e multiplicada pelo peso correspondente. Isso permite que o modelo busque padrões de consumo similares baseados apenas na idade do usuário.

Essa abordagem garante que mesmo usuários sem histórico consigam receber sugestões plausíveis, comparando seu perfil com usuários de idade similar presentes nos dados de treinamento.

Realizando a predição para cada produto

Com o vetor do usuário gerado, iniciamos a etapa de recomendação. Para isso, percorremos todos os produtos e concatenamos o vetor do usuário com o vetor de cada produto, formando pares de entrada. Esses pares são agrupados em um tensor 2D, representando o conjunto de entradas para predição.

Esse tensor é então processado pela rede neural treinada, que retorna um vetor com as probabilidades de compra para cada produto. Esses valores representam o grau de compatibilidade entre o usuário e cada item do catálogo.

Interpretando os resultados da predição

Cada valor retornado pelo modelo indica a confiança na recomendação de um produto específico. Por exemplo:

- Valor próximo de 1: alta probabilidade de compra
- Valor próximo de 0: baixa probabilidade de compra

Com os valores em mãos, combinamos os metadados dos produtos com os scores de predição, criando um objeto com nome, atributos e grau de

recomendação. Em seguida, ordenamos os produtos do mais recomendado para o menos recomendado.

Enviando as recomendações para a interface

Depois de processar os resultados, utilizamos `postMessage` para enviar as recomendações de volta à interface do usuário. Essa comunicação permite que a interface atualize dinamicamente as sugestões com base no perfil de cada usuário selecionado.

O sistema é capaz de responder de forma instantânea, mesmo para usuários com poucos dados, o que demonstra a eficiência do modelo treinado e do código implementado em JavaScript com TensorFlow.js.

Considerações sobre escalabilidade

Durante a aula, discutimos como este sistema se comportaria em ambientes com grandes volumes de dados. Como estamos comparando todos os produtos com todos os usuários, isso se torna inviável com milhões de registros.

A solução apontada é a utilização de bancos de dados vetoriais como Pinecone, ChromaDB ou extensões do PostgreSQL, que permitem indexar vetores e realizar buscas aproximadas de alta performance. Assim, ao invés de comparar com todos os produtos, o sistema busca os "top N" mais próximos e realiza as previsões apenas nesses casos, otimizando recursos.

Avaliando os resultados práticos

Testes realizados com usuários reais mostraram a eficácia do modelo. Por exemplo, ao selecionar um usuário de 30 anos sem compras, o sistema sugeriu produtos comprados por outros usuários com idade semelhante. Em outros casos, ao adicionar um produto ao carrinho, a lista de recomendações era reordenada automaticamente para refletir a nova preferência detectada.

Conclusão do projeto

Com isso, finalizamos a implementação completa do sistema de recomendação: desde a leitura e preparação dos dados, codificação de produtos e usuários, treinamento da rede neural, até a predição e exibição de recomendações.

A proposta foi mostrar que é possível desenvolver um sistema funcional e didático usando apenas JavaScript e TensorFlow.js, com todo o processamento feito no navegador. Isso é ótimo para protótipos, testes e aplicações leves.

Agora, cabe a você adaptar esse conhecimento para outros contextos, como recomendação de filmes, artigos ou qualquer outro tipo de item. Basta ajustar as features e seguir os mesmos princípios.

Missão cumprida!

Capítulo 7: Como funcionam Sistemas de recomendação PT07

Finalizamos nosso módulo com um convite prático: aplicar os conhecimentos adquiridos em um projeto real. Com o sistema de recomendação já implementado e funcional, chegou a hora de explorar possibilidades e ampliar o impacto do que foi aprendido.

Projetos práticos e desafios

Minha sugestão é que você crie um projeto pessoal usando essa mesma arquitetura de recomendação que construímos durante o módulo. Pode ser, por exemplo, uma aplicação que recomenda filmes, produtos, livros, cursos ou qualquer outro tipo de item. O importante é aplicar a lógica de representação de dados, codificação de usuários e itens, treinamento de rede neural e uso do modelo para gerar recomendações.

Use o mesmo código base e adapte conforme o novo contexto. Isso vai consolidar o aprendizado e ainda gerar um projeto que você pode mostrar para a comunidade e usar como portfólio profissional.

Compartilhamento com a comunidade

Depois de finalizar o seu projeto, compartilhe no nosso canal do Discord. Lá, você poderá trocar ideias com outros alunos, receber feedbacks e até colaborações para evoluir o sistema.

Além disso, publique também nas suas redes sociais. Mostrar que você não apenas estudou, mas aplicou na prática, é algo que gera autoridade e reconhecimento profissional. E se você me marcar, eu mesmo vou compartilhar seu projeto para ajudar a dar mais visibilidade ao seu trabalho.

Usando banco de dados vetoriais para produção

Para quem quiser levar o projeto para um nível mais próximo de produção, recomendo experimentar um banco de dados vetorial como Pinecone, ChromaDB ou até mesmo a extensão vetorial do PostgreSQL. Com isso, você pode armazenar os vetores gerados do modelo (usuários e produtos) diretamente no banco, e realizar buscas por similaridade de forma escalável.

A ideia é substituir a comparação com todos os produtos por uma busca pelos "top N" produtos mais próximos do vetor do cliente. Assim, você otimiza performance e torna o sistema viável para uso com grandes volumes de dados.

Na hora de realizar a predição, basta recuperar os produtos mais próximos do banco e aplicar a função `predict` da rede neural, gerando assim as recomendações personalizadas com base em dados reais.

Encontrando bases reais para treinamento

Se você não quiser montar uma base de dados manualmente, recomendo explorar o Kaggle. Lá você encontra dezenas de bases prontas com milhares de registros, ideais para testar seu sistema com dados reais.

Com isso, é possível ir além de um protótipo e experimentar comportamentos de produção em ambiente controlado.

Fechamento do módulo

Espero que este módulo tenha superado suas expectativas. Meu objetivo foi te mostrar, passo a passo, como construir um sistema de recomendação do zero,

no navegador, com TensorFlow.js, integrando conceitos de representação vetorial, redes neurais e interação em tempo real.

Agora, cabe a você aplicar, testar, adaptar e evoluir esse conhecimento nos seus projetos pessoais e profissionais. Você tem tudo o que precisa para ir mais longe.

Te vejo no próximo módulo!

Módulo 04 : Web Machine Learning - Como Vencer Qualquer Jogo

Capítulo 1: Como Vencer Qualquer Jogo PT01

Neste novo módulo, iniciamos um projeto completamente diferente dos anteriores: criar uma inteligência artificial que joga sozinha um jogo clássico de tiro, o Duck Hunt, totalmente no navegador. A proposta é mostrar como usar modelos de detecção de objetos prontos para automatizar tarefas complexas, como vencer um jogo.

Mudança de foco: integração em vez de treinamento

Ao contrário dos módulos anteriores, onde treinamos modelos do zero, aqui o foco é utilizar modelos já treinados, Web APIs e bibliotecas próprias do JavaScript. A ideia é mostrar que nem sempre precisamos construir tudo do início. Com o conhecimento certo, podemos integrar soluções existentes e aplicá-las de forma criativa e eficaz.

O desafio: automatizando o Duck Hunt

Inspirado por um jogo da minha infância, o Duck Hunt, resolvi explorar como ensinar um programa a jogar sozinho. Minha primeira tentativa foi criar uma rede neural para identificar os patos e efetuar disparos. No entanto, tive dificuldades com a acurácia e a complexidade do treinamento.

Foi então que entrei em contato com Hugo Zannini, especialista em Machine Learning e reconhecido como Google Developer Expert, que sugeriu uma abordagem mais eficiente usando um modelo pronto: o YOLO (You Only Look Once).

O modelo YOLO

O modelo YOLO é amplamente utilizado para detecção de objetos. Ele analisa uma imagem e retorna todos os objetos identificados com suas respectivas coordenadas. Apesar de ter sido treinado originalmente em Python com PyTorch, ele pode ser convertido para TensorFlow.js e executado no navegador, o que o torna ideal para este projeto.

O modelo reconhece mais de 80 categorias de objetos, como carros, bicicletas, animais, malas e equipamentos esportivos. No nosso caso, ao enviar imagens do Duck Hunt, o modelo identificou os patos como "kite" (pipa). Embora não seja a classificação perfeita, foi suficiente para nosso objetivo.

Integração com o jogo

Com o modelo funcionando, o processo se tornou simples:

1. Capturamos uma imagem do jogo.
2. Enviamos essa imagem para o modelo YOLO.
3. Recebemos as coordenadas dos objetos detectados (os "kites").
4. Enviamos essas coordenadas de volta ao jogo.
5. A inteligência artificial efetua o disparo automaticamente.

Essa integração mostra o poder da combinação entre modelos prontos e interação em tempo real com sistemas visuais.

Sobre o jogo Duck Hunt em JavaScript

Para facilitar a demonstração, usei um fork de um projeto open source criado por Matt, que reimplementou o Duck Hunt em JavaScript usando a biblioteca Pixi.js para renderizações 2D.

No momento da gravação, o projeto só executava em versões antigas do Node.js (como a 8), pois estava sem atualizações há mais de cinco anos. Atualizei todas as dependências, ajustei o que era necessário e criei um pull request com diversas melhorias para garantir que o jogo funcionasse corretamente nos ambientes atuais.

Adicionalmente, estendi o jogo com uma mira visual, permitindo que vejamos exatamente onde a IA está mirando e atirando.

Preparando o ambiente para a prática

Para facilitar o aprendizado, já deixei um template pronto para uso. Ele contém:

- O jogo ajustado e atualizado
- O modelo YOLO já baixado e pronto para uso
- As estruturas de código organizadas para que você foque apenas na parte de detecção de objetos

Basta baixar o template, abrir no seu editor de código e acompanhar a demonstração. Você entenderá como integrar o modelo, capturar os alvos e automatizar os disparos.

Conclusão da primeira parte

Este primeiro contato com o projeto mostra que é possível utilizar ferramentas e modelos já existentes para criar soluções inovadoras e funcionais. Com apenas JavaScript e TensorFlow.js, conseguimos implementar uma IA que joga um game clássico sozinha, de forma eficiente e didática.

Na próxima parte, vamos detalhar como o modelo recebe as imagens, interpreta os objetos e como realizamos os disparos no código.

Vamos em frente!

Capítulo 2: Como Vencer Qualquer Jogo PT02

Dando continuidade ao nosso projeto de construir uma IA que joga o Duck Hunt automaticamente, neste capítulo exploramos a estrutura do código, a organização das threads e como ocorre o disparo automático com base nas predições do modelo de detecção de objetos.

Estrutura do projeto

Começamos organizando o código em uma nova pasta chamada `exemplo02`, onde colocamos o modelo YOLO já baixado, o jogo atualizado e o layout com o marcador de pontuação.

Antes de executar o projeto, garantimos que a versão correta do Node.js está instalada, utilizando o comando `npm ci` para restaurar as dependências. No `package.json`, o script utilizado é apenas o `start`, que executa o jogo com as alterações que fizemos.

Comportamento inicial da IA

Assim que o jogo é iniciado, vemos que a IA está disparando sem controle e errando todos os tiros. Isso acontece porque a estrutura está montada, mas o processo de detecção e acerto ainda não está completamente funcional. A cada nova predição do modelo, a mira é atualizada para apontar para a posição do pato detectado e um disparo é simulado, atualizando também o marcador de pontos.

Integração com a interface do jogo

As alterações feitas no layout do jogo incluem uma mira visual e uma interface de predições. A mira é inicialmente invisível e só aparece ao receber coordenadas de um pato detectado. A comunicação com a IA ocorre através de uma *Web Worker*, que é uma thread secundária executando o modelo.

Captura da imagem do jogo

Como o jogo é renderizado usando Pixi.js sobre um elemento `canvas`, utilizamos a API `createImageBitmap` do JavaScript para capturar um print da tela. Essa

imagem é enviada à thread secundária (worker), que processa a imagem com o modelo de detecção de objetos e retorna as coordenadas do alvo.

A thread principal, por sua vez, nunca processa diretamente os dados do modelo. Ela apenas coleta as informações, envia ao worker e aguarda o retorno para atualizar a interface.

Funcionamento do Worker

No worker, importamos o TensorFlow.js e o modelo YOLO. Embora o modelo esteja carregado, ainda não foi inicializado com as imagens. Quando a thread recebe uma mensagem do tipo `predict`, ela executa o processamento da imagem e retorna as coordenadas X e Y do pato, bem como o score de confiança da predição.

Para referência, também foi incluído um arquivo `labels.json` com todas as categorias que o modelo é capaz de identificar. Isso permite testar o modelo com outras imagens ou jogos, ampliando as possibilidades de aplicação.

Simulando o clique no jogo

Ao retornar os dados do worker, a thread principal atualiza a posição da mira no jogo e simula um clique para atingir o pato detectado. Isso cria um comportamento que imita um jogador humano interagindo com o mouse. A cada nova predição, um novo disparo é feito, criando um ciclo de detecção e ação totalmente automatizado.

Otimização gráfica do navegador

Durante os testes, foi observado que o jogo pode apresentar falhas se a aceleração gráfica do navegador estiver desabilitada. Para evitar isso, é necessário manter a opção "Use Graphics Acceleration When Available" ativada nas configurações do navegador.

Essa configuração garante que os elementos renderizados no `canvas` sejam processados corretamente, permitindo que o jogo e a IA funcionem sem travamentos ou falhas de renderização.

Considerações finais

Com essa estrutura pronta, o sistema já é capaz de detectar os patos no jogo, posicionar a mira e disparar automaticamente. Essa integração entre TensorFlow.js, Web Workers e APIs gráficas do navegador mostra como é possível criar experiências interativas e inteligentes com tecnologias acessíveis.

No próximo capítulo, vamos aprimorar ainda mais essa IA, ajustando o modelo, refinando os disparos e explorando novos comportamentos para aumentar a precisão da nossa inteligência artificial.

Vamos em frente!

Capítulo 3: Como Vencer Qualquer Jogo PT03

Nesta terceira parte da nossa jornada para automatizar o jogo Duck Hunt com uma IA, focamos na conversão da imagem capturada em um tensor compatível com o modelo YOLO, e em como processar os dados retornados pela inferência para preparar as próximas etapas da identificação de objetos e tomada de decisão.

Carregando e aquecendo o modelo

O primeiro passo desta aula foi garantir que o TensorFlow.js estivesse pronto. Utilizamos a função `tf.ready()` para aguardar essa inicialização e em seguida carregamos dois elementos principais: o arquivo de labels (categorias reconhecíveis pelo modelo) e o próprio modelo YOLO, usando `tf.loadGraphModel`.

Após o carregamento, realizamos o "aquecimento" do modelo, ou seja, executamos uma predição inicial com um tensor de entrada preenchido com valores fixos. Isso melhora o desempenho das predições subsequentes, pois permite que o modelo carregue e mantenha em cache os componentes internos necessários para as inferências.

Convertendo imagem para tensor

Com o modelo carregado, passamos a trabalhar na função `preprocessImage`, que transforma uma imagem capturada do jogo em um tensor adequado para inferência.

Essa conversão é feita com `tf.browser.fromPixels`, que transforma um bitmap em um tensor, seguido de `tf.image.resizeBilinear` para redimensionar a imagem para 640x640 pixels, dimensão exigida pelo modelo YOLO.

Após isso, normalizamos os valores dos pixels dividindo por 255, convertendo para a faixa de 0 a 1. Por fim, adicionamos uma dimensão de "batch" usando `expandDims`, para tornar o tensor compatível com o modelo.

Envolvemos todo esse processo dentro de `tf.tidy`, uma boa prática que assegura a liberação automática da memória dos tensores criados temporariamente.

Executando a inferência

Com a imagem convertida em tensor, criamos a função `runInference`, que chama `model.executeAsync(input)` para processar a imagem e retornar os resultados da detecção.

Essa chamada retorna um array de saídas, das quais nos interessamos pelas três primeiras:

`boxes`: coordenadas das caixas delimitadoras dos objetos

`scores`: confiança das detecções

`classes`: categorias detectadas

Transformamos essas saídas em arrays JavaScript com `data()` e utilizamos `Promise.all` para processar todas simultaneamente. Em seguida, liberamos os tensores retornados usando `dispose`, mantendo a eficiência de memória.

Validando os resultados

Com os dados em mãos, fizemos testes de validação para garantir que os valores retornados estivessem coerentes. Os arrays `boxes`, `scores` e `classes` foram inspecionados e confirmamos que o modelo retornava informações a cada frame, mesmo que ainda não compreensíveis de imediato.

Os valores nos arrays `scores` indicam o grau de confiança de que um objeto foi detectado corretamente. Os `boxes` trazem as posições relativas no frame da imagem onde o objeto foi identificado. E `classes` informa qual categoria foi atribuída a cada detecção.

Pronto para o pós-processamento

Ao final desta etapa, temos os dados brutos retornados pela inferência prontos para serem filtrados, classificados e utilizados na tomada de decisão do jogo. Ainda que esses dados ainda precisem de interpretação para que a IA saiba onde mirar e atirar, eles representam o coração da inteligência artificial responsável pelas ações automatizadas.

Na próxima parte, vamos criar as funções de pós-processamento desses dados para escolher os melhores alvos com base na confiança e precisão das detecções, concluindo assim o sistema de tomada de decisão da nossa IA.

Vamos em frente!

Capítulo 4: Como Vencer Qualquer Jogo PT04

Nesta última parte do projeto, finalizamos a criação da inteligência artificial que joga Duck Hunt automaticamente, transformando os dados brutos de inferência do modelo em coordenadas reais da tela, filtrando as detecções com base na confiança e convertendo as caixas delimitadoras em pontos precisos para realizar os disparos.

Transformando as predições em coordenadas

Começamos com a função `processPrediction`, que recebe os resultados de inferência (`boxes`, `scores` e `classes`), a largura (`width`) e altura (`height`) da tela. Essa função percorre os arrays retornados pelo modelo usando o mesmo índice para associar corretamente cada caixa com seu score e classe.

Implementamos uma função geradora que, para cada objeto identificado com confiança suficiente, retorna um objeto contendo a posição X e Y do centro da caixa delimitadora, além do score da predição.

Filtro por confiança e classe

Para garantir a precisão dos disparos, utilizamos um limite de confiança (`classThreshold`) de 40%. Valores acima desse limite são considerados válidos. Também filtramos os resultados para incluir apenas detecções da classe "kite", que foi como o modelo interpretou os patos do jogo. Se a classe detectada for diferente, ela é ignorada.

Cálculo do centro da caixa delimitadora

A cada item aceito, extraímos suas coordenadas normalizadas e convertidas para pixels reais com base no `width` e `height` da tela. A caixa é definida pelas coordenadas `x1`, `y1`, `x2`, `y2`, e o centro é calculado com base na diferença entre essas posições:

- $\text{centerX} = x1 + (x2 - x1) / 2$
- $\text{centerY} = y1 + (y2 - y1) / 2$

Essas coordenadas são então usadas como destino para o disparo automatizado da IA.

Disparos automáticos e desempenho

Cada objeto processado é enviado para a thread principal por meio de `postMessage`, que aciona a mira da IA e dispara no ponto identificado. Durante os testes, foi possível acompanhar no console as predições com coordenadas e confiança.

Observamos que a IA acertava a maioria dos patos, mesmo com pequenas imprecisões. O modelo conseguiu zerar o jogo, passando por todos os níveis, mesmo com condições mais complexas (como maior quantidade de patos por fase).

Possíveis melhorias

Com o sistema funcionando, podemos refinar ainda mais o comportamento da IA:

- Ajustar dinamicamente o `classThreshold` para aumentar a precisão.
- Controlar melhor o intervalo entre capturas de tela (além do simples uso de `setTimeout`).
- Implementar filtros adicionais para ignorar detecções com baixa confiança ou fora da área jogável.
- Estender para outros jogos ou contextos, mudando apenas a interpretação dos labels e o alvo dos disparos.

Fechamento do projeto

Com isso, encerramos o projeto prático do módulo, onde mostramos como:

1. Capturar a imagem do jogo em tempo real
2. Processar com um modelo YOLO no navegador
3. Filtrar, converter e usar as predições para criar uma IA que interage com o jogo

Esse tipo de experiência mostra como é possível usar IA com ferramentas acessíveis, sem depender de servidores externos ou hardwares especializados. Tudo rodando no navegador, com código aberto e didática clara.

Agora é com você. Adapte, experimente, melhore e compartilhe com a comunidade. Te vejo no próximo módulo!

Capítulo 5: Como Vencer Qualquer Jogo PT05

Encerrando nosso módulo sobre a criação de uma IA que joga sozinha o Duck Hunt, quero deixar um convite prático, um desafio e uma inspiração.

Um projeto de brincadeira, mas com aplicações reais

Embora este projeto tenha um tom lúdico, ele demonstra um conceito extremamente poderoso: a possibilidade de usar modelos prontos de visão computacional para automatizar qualquer tarefa visual. A ideia de "vencer um jogo" é apenas um exemplo divertido de como a detecção de objetos pode ser usada para interagir com sistemas visuais em tempo real.

Como vimos ao longo das aulas, capturamos imagens da tela, processamos com o modelo YOLO diretamente no navegador e usamos os resultados para tomar decisões automáticas. Essa mesma abordagem pode ser aplicada em sistemas de monitoramento, robótica, navegação autônoma e outras tantas áreas.

Desafio prático: hackeie outro jogo

Meu desafio para você é o seguinte: encontre outro jogo que rode no navegador, de preferência simples e com elementos visuais claros, e tente criar uma IA que o jogue sozinha. Pode ser um jogo de pulo, de corrida ou quebra-cabeça. O objetivo é que você aplique os mesmos conceitos:

- Captura da tela com `createImageBitmap`
- Processamento com modelo YOLO
- Conversão das predições em coordenadas
- Geração de ações automatizadas

Esse desafio vai consolidar de vez o que você aprendeu aqui.

Estendendo o modelo: mais classes, outros objetos

Como tarefa extra, recomendo que você explore o artigo do Hugo Zannini (link fornecido na aula). Nele, Hugo mostra como estendeu o modelo YOLO para detectar prateleiras vazias em supermercados. Ao adicionar novas classes e

treinar o modelo com outros exemplos, ele conseguiu adaptar o modelo para um problema específico.

Minha sugestão é que você repita esse processo, mas com objetos diferentes. Pode ser, por exemplo, identificar peças de Lego, lixos recicláveis, placas de trânsito ou qualquer outro item de interesse pessoal ou profissional.

Um mundo de possibilidades

Com o que você aprendeu neste módulo, abre-se um leque de possibilidades. Você agora tem capacidade de:

- Capturar e interpretar informações visuais
- Executar IA diretamente no navegador
- Integrar modelos com sistemas interativos em tempo real
- Criar soluções criativas e funcionais usando JavaScript e TensorFlow.js

Isso vale ouro em qualquer cenário profissional onde automação e visão computacional são relevantes.

Compartilhe sua ideia e inspire outros

Quero te convidar a deixar nos comentários da aula que ideias de projetos você teve com base neste módulo. Não importa se é algo simples ou ambicioso. Compartilhar seu pensamento é um passo importante para inspirar outros, receber feedbacks e tornar sua ideia real.

Conclusão do módulo

Espero que este módulo tenha superado suas expectativas. Começamos com um jogo simples, mas terminamos com um sistema completo de IA interativa no navegador. Você viu, na prática, como modelos prontos podem ser usados de forma criativa, eficiente e produtiva.

Agora é a sua vez. Pratique, explore, adapte, expanda. Nos vemos no próximo módulo!

Módulo 05: Inteligência artificial na Web

Capítulo 1: Algoritmos Genéticos

Neste capítulo, exploramos os algoritmos genéticos como uma forma alternativa e fascinante de aprendizado de máquina. Esses algoritmos são inspirados na evolução biológica e são uma excelente maneira de encontrar soluções criativas para problemas complexos, especialmente quando não se tem uma resposta clara de como resolver determinada tarefa.

Inteligência artificial além do básico

Durante o módulo, vimos como usar IA para automatizar jogos como o Duck Hunt. Mas ali, a IA apenas reconhecia objetos na tela e apontava onde achava que estava o alvo. Não havia aprendizado de verdade, apenas reconhecimento. Para que uma IA realmente aprenda a vencer um jogo, entramos em outro tipo de abordagem: o aprendizado por reforço.

Essa é uma forma poderosa de aprendizado em que o algoritmo aprende por tentativa e erro, sendo recompensado por acertos e penalizado por erros. Um exemplo clássico é o jogo da cobrinha, onde a IA ganha pontos ao comer frutas e perde ao colidir com a parede ou com o próprio corpo. A cada movimento, ela recebe uma pontuação e aprende com isso.

O que são algoritmos genéticos

Os algoritmos genéticos seguem uma lógica diferente. Em vez de aprender sequencialmente, eles trabalham com uma população de soluções candidatas. Cada solução é avaliada, e as melhores são selecionadas para gerar novas soluções por meio de cruzamento e mutação.

Imagine que você é um inventor tentando criar a melhor roda para um carro. Você gera vários carros com formas e tamanhos diferentes e os coloca para rodar em uma pista. Aqueles que andam mais longe, mais rápido e com mais estabilidade são considerados melhores. Então você mistura as características

dos melhores carros e gera uma nova geração de carros. Esse processo é repetido várias vezes, com pequenas mutações introduzidas a cada nova geração.

Simulações e criatividade inesperada

O resultado desse processo pode ser surpreendente. Os carros gerados muitas vezes possuem formas incomuns que um humano jamais teria projetado, mas funcionam incrivelmente bem dentro dos critérios de avaliação. Isso acontece porque a evolução prioriza desempenho, não estética.

Na prática, você pode testar isso em simuladores que rodam no próprio navegador, onde é possível acompanhar a evolução de geração em geração. A cada nova geração, os resultados melhoram, os "carros" se adaptam melhor à pista, e o algoritmo vai encontrando soluções mais eficientes sem nenhuma instrução direta de como fazer isso.

Parâmetros e ajustes

Durante a simulação, você pode controlar parâmetros como taxa de mutação, gravidade, formato do terreno e outros fatores. Esses ajustes impactam diretamente a evolução dos indivíduos. Uma taxa de mutação muito alta pode gerar resultados imprevisíveis e ineficazes, enquanto uma mutação muito baixa pode causar estagnação, sem inovação.

O segredo está no equilíbrio: mutações suficientes para explorar novas possibilidades, mas com controle para manter o que já funciona bem. Esse tipo de ajuste fino é essencial em projetos de IA evolutiva.

Casos de uso e inspiração

O uso de algoritmos genéticos vai muito além de jogos. Eles são aplicáveis em problemas de engenharia, logística, design de circuitos, otimização de processos e muito mais. Existem casos em que AIs aprenderam a estacionar carros sozinhas, inclusive fazendo balizas com drift, tudo graças a esse tipo de algoritmo.

Outro exemplo inspirador foi apresentado em uma conferência no Brasil, onde um algoritmo genético foi usado para ensinar uma IA a jogar o jogo do dinossauro do Chrome. A IA aprendeu a pular e se esquivar dos obstáculos sozinha, sem nenhuma instrução específica. Isso demonstra o potencial criativo dos algoritmos genéticos.

Diferenças com aprendizado por reforço

A principal diferença entre os algoritmos genéticos e o aprendizado por reforço está na forma como aprendem. O aprendizado por reforço envolve um agente que toma decisões sequenciais para maximizar uma recompensa acumulada ao longo do tempo. Já os algoritmos genéticos lidam com populações inteiras de soluções, avaliando, cruzando e mutando indivíduos até chegar à melhor solução.

Enquanto um foca na tomada de decisão passo a passo, o outro foca na evolução de candidatos simultaneamente.

Considerações finais

Com os algoritmos genéticos, você viu como é possível aplicar princípios de evolução natural para resolver problemas de forma criativa e eficiente. Trata-se de um campo fascinante que, mesmo com conceitos simples, oferece resultados incríveis e aplicações reais no mundo da tecnologia.

Recomendo que você explore os links indicados na aula e experimente ferramentas interativas baseadas nesses algoritmos. Brinque com os parâmetros, observe os resultados e imagine como essas técnicas poderiam ser aplicadas em problemas do seu dia a dia ou em desafios do mercado.

Nos vemos na próxima aula!

Capítulo 2: Como funcionam LLMs: transformers, embeddings, attention

Neste capítulo, mergulhamos nos fundamentos das Large Language Models (LLMs), explicando como funcionam internamente tecnologias como o ChatGPT, Gemini e outras. O foco aqui é desvendar os principais componentes que tornam essas IAs tão poderosas: tokenização, embeddings, arquitetura transformer, attention e sampling.

O que são LLMs?

LLM é a sigla para "Large Language Model" ou Modelo de Linguagem de Grande Escala. Esses modelos são treinados com quantidades massivas de texto para aprender padrões da linguagem humana e gerar respostas coerentes. O termo GPT, como em ChatGPT, significa "Generative Pre-trained Transformer", e cada parte dessa sigla tem papel fundamental:

- Generative: o modelo é gerador de texto.
- Pre-trained: antes de responder perguntas, ele é treinado com muitos textos.
- Transformer: é a arquitetura de rede neural que torna tudo isso possível.

1. Tokenização

Antes de um texto ser processado, ele é quebrado em unidades menores chamadas *tokens*. Esses tokens não são necessariamente palavras; podem ser partes de palavras, espaços, sinais de pontuação, etc. O modelo trabalha com esses tokens em vez de palavras completas, pois são mais eficientes para representar a linguagem.

Um ponto importante: uma palavra pode conter vários tokens. Por isso, quando um modelo limita um prompt a 4.000 tokens, isso não significa 4.000 palavras, e sim unidades menores.

2. Embeddings

Uma vez que o texto é tokenizado, cada token é transformado em um vetor de números. Esses vetores, chamados de *embeddings*, representam o significado das palavras no contexto.

Palavras que aparecem em contextos semelhantes acabam com embeddings semelhantes. Por exemplo:

- "Carro" e "veículo" costumam aparecer em frases similares e têm embeddings próximos.
- "Médico" e "hospital" não são sinônimos, mas aparecem juntos com frequência, e por isso têm relação semântica.

Essas representações vetoriais permitem que o modelo compreenda nuances como:

- Rei - Homem + Mulher = Rainha
- Paris - França + Itália = Roma

Esses cálculos funcionam porque relações semânticas recorrentes são transformadas em direções no espaço vetorial.

3. Transformers e o mecanismo de Attention

Com os tokens convertidos em embeddings, o próximo passo é processar essas informações com a arquitetura *Transformer*. Essa é uma das maiores revoluções da IA moderna.

O Transformer utiliza um mecanismo chamado *Self-Attention*, que permite ao modelo considerar todas as palavras do contexto ao interpretar ou gerar cada token. Por exemplo, na frase:

"A Maria contou para a Ana que ela foi promovida."

O token "ela" pode se referir tanto à Maria quanto à Ana. O mecanismo de Attention ajuda o modelo a determinar qual referência faz mais sentido, considerando o contexto completo.

O Transformer também corrige limitações de modelos antigos, como a dificuldade em lidar com frases longas e o processamento sequencial. Ele processa todos os tokens em paralelo e entende relações de longa distância.

Adicionalmente, o modelo adiciona *positional encodings* aos embeddings, já que os vetores por si só não carregam informação de ordem. Assim, frases como "o cachorro mordeu o homem" e "o homem mordeu o cachorro" são corretamente interpretadas.

4. Probabilidades e Decoding

Depois que o Transformer processa os embeddings, ele gera uma lista de probabilidades para os próximos tokens possíveis.

Por exemplo, se a frase for:

"O céu é..."

O modelo pode atribuir:

- azul: 55%
- nublado: 18%
- claro: 10%
- bonito: 7%

Com base nessas probabilidades, o modelo escolhe um token. O método de escolha depende de parâmetros como:

- Temperature: define o grau de aleatoriedade. Valores baixos geram respostas mais determinísticas (menos criativas), valores altos aumentam a variedade.
- Top-K: limita a escolha aos K tokens mais prováveis.
- Top-P (nucleus sampling): soma as probabilidades até atingir um limiar, como 90%, e escolhe dentro desse subconjunto.

Esses parâmetros influenciam diretamente a qualidade e variedade das respostas geradas pela IA.

5. Geração passo a passo (Sampling)

A IA não gera todo o texto de uma vez. Ela gera token por token, recalculando o contexto a cada nova palavra. Esse processo se chama *sampling*.

A cada nova iteração:

1. A IA analisa o texto gerado até o momento.
2. Calcula a probabilidade dos próximos tokens.
3. Escolhe um novo token.
4. Adiciona esse token ao contexto.

Esse ciclo se repete até atingir o fim da resposta. Quanto maior o texto, maior o custo computacional.

Alucinações e Limitações

O modelo não sabe o que é verdade ou mentira. Ele apenas gera o token mais provável dado o contexto. Quando falta informação ou o prompt é ambíguo, ele pode gerar afirmações falsas que soam convincentes.

Para reduzir essas *alucinações*, é importante:

- Fornecer contexto completo.
- Permitir respostas do tipo "não sei".
- Evitar prompts que pressionem por certeza absoluta.

Considerações finais

Entender como funcionam as LLMs ajuda a tirar melhor proveito delas, tanto na escrita de prompts quanto na interpretação dos resultados. Conhecendo os mecanismos internos como tokenização, embeddings, attention e sampling, você consegue:

- Reduzir custos otimizando prompts.
- Ajustar parâmetros para obter resultados mais precisos ou criativos.
- Criar experiências mais robustas com APIs de IA.

Com esse conhecimento, você está pronto para avançar para experimentações práticas e aplicações mais sofisticadas. Te vejo na próxima aula!

Capítulo 3: Como funciona inteligencia artificial na Web com Web AI

Neste capítulo, exploramos como a inteligência artificial está sendo integrada diretamente nos navegadores, permitindo que modelos de linguagem e outras funcionalidades avançadas sejam executados localmente na máquina do usuário. Trata-se de uma mudança de paradigma significativa, abrindo caminho para a chamada Web 4.0.

A Web 4.0 e a IA no navegador

A chamada Web 4.0 representa uma nova geração da internet, na qual a inteligência artificial é nativa ao navegador. Isso significa que, em vez de depender de servidores remotos para processar IA, as aplicações podem rodar diretamente na máquina do usuário. Modelos como o Gemma do Google e o DeepSeek já estão sendo utilizados dessa forma.

Essa abordagem traz benefícios claros:

- Não é necessário enviar dados dos usuários para servidores externos.
- A performance é aceitável, mesmo em aplicações mais complexas.
- A execução local garante maior privacidade.

Desvantagens e desafios

Apesar das vantagens, ainda existem entraves significativos:

- Os modelos são pesados: o Gemma possui cerca de 2.5GB e o DeepSeek, 1.3GB.
- O tempo de download pode comprometer a experiência, especialmente em dispositivos móveis.
- Nem todo navegador está preparado para essa funcionalidade de forma nativa.

APIs experimentais e o futuro

O Google vem investindo em APIs experimentais que buscam integrar modelos como o Gemini Nano diretamente ao navegador. Com isso, ao instalar o navegador, o modelo já viria embarcado, eliminando a necessidade de downloads futuros. Algumas dessas APIs já estão disponíveis em versões do Chrome e do Chrome Canary.

Funções já disponíveis:

- Tradução de texto.
- Identificação de idioma.
- Resumo de conteúdo.
- Prompt para interação com LLMs.

Todas essas funcionalidades já podem ser executadas localmente, gratuitamente e com bom desempenho.

Exemplo prático: executando uma LLM no navegador

A demonstração criada nesta aula mostra como é simples utilizar uma LLM diretamente no navegador, utilizando apenas um arquivo HTML com JavaScript.

O processo é:

1. Habilitar as funcionalidades experimentais no navegador.
2. Criar um prompt inicial com contexto.
3. Configurar parâmetros como `temperature` e `topK` para controlar a criatividade e variedade das respostas.
4. Executar a API `languageModel.create()` passando os parâmetros.
5. Processar a resposta usando streaming token por token.

Esse modelo responde diretamente dentro do navegador, sem conexão com servidores externos, e permite experimentar diferentes comportamentos conforme a temperatura e o topK escolhidos.

Testando diferentes configurações

Ao manipular os parâmetros `temperature` e `topK`, é possível observar a diferença no comportamento da IA:

- Com **temperature** próximo de 0, a resposta tende a ser mais previsível.
- Com valores mais altos, a IA tende a ser mais criativa.
- O **topK** define quantos candidatos ao próximo token são considerados.

Por exemplo:

- Pergunta: "O céu é..."
- Com **temperature=0**, a resposta quase sempre será "azul".
- Com **temperature=2** e **topK=10**, a resposta pode variar para "vasto", "ilimitado", "cheio de estrelas", etc.

Validação de similaridade semântica

Relembrando conceitos anteriores, também foi testada a capacidade de razão semântica do modelo, por exemplo:

- Prompt: "Rei - homem + mulher = ?"
- Resultado: "Rainha"

Esse tipo de raciocínio mostra como os embeddings estão funcionando por trás da IA, mesmo no ambiente do navegador.

Conclusão

Com os avanços em Web AI, estamos entrando em uma nova era da computação, onde IA não depende mais da nuvem. Modelos embutidos nos navegadores oferecem maior controle, privacidade e independência.

A Web 4.0, como foi chamada aqui, é a visão de um futuro onde os navegadores se tornam os verdadeiros sistemas operacionais, capazes de executar IA de forma nativa, gratuita e descentralizada. Se você é desenvolvedor, agora é a hora de experimentar e integrar essas ferramentas aos seus projetos.

Capítulo 4: Demo 02 - Web AI Multi Modal

Neste capítulo, mergulhamos nas capacidades multimodais das novas APIs de Web AI. O foco é demonstrar como modelos de linguagem embarcados nos navegadores podem agora lidar com diferentes tipos de entrada, como texto, áudio e imagem, tudo rodando localmente, sem conexão com servidores externos.

A Web de agentes inteligentes

Estamos presenciando o nascimento da Web 4.0, onde agentes inteligentes embarcados nos navegadores serão parte integrante da experiência digital. A ideia é simples e revolucionária: permitir que os sites ofereçam assistentes inteligentes que interajam diretamente com os visitantes, sem a necessidade de navegar por várias páginas ou enviar dados para servidores remotos.

A Google está liderando essa transformação com APIs que possibilitam a execução local de modelos como o Gemini Nano. E, recentemente, essas APIs passaram a aceitar entradas multimodais.

O que é multimodalidade?

Multimodalidade é a capacidade de um sistema de IA de lidar com diferentes tipos de dados: texto, imagem, áudio, entre outros. A IA não apenas entende esses formatos, mas também é capaz de relacionar conteúdos entre eles.

Por exemplo:

Enviar uma imagem e perguntar o que está nela.

Enviar um áudio e pedir a transcrição.

Combinar texto e imagem no mesmo prompt para uma resposta contextualizada.

Demonstração prática

Na demonstração feita em aula, um projeto chamado "Exemplo 05 - Web AI Multimodal" foi apresentado. Esse projeto é capaz de:

Processar imagens de pessoas, animais e cenários.

Descrever conteúdos das imagens com alto grau de precisão.

Transcrever documentos, como um cartão CNPJ, para texto.

O mais impressionante é que tudo isso ocorre 100% offline, no navegador do usuário.

Como funciona

O funcionamento da IA multimodal depende de algumas etapas:

Verificação do ambiente: o navegador precisa ser compatível (por enquanto, apenas Chrome).

Inicialização do modelo: assim que a página carrega, o modelo é baixado localmente.

Recebimento de inputs: imagem, áudio ou texto.

Processamento do dado: o arquivo é convertido em formato binário (blob).

Geração de resposta: a IA interpreta os dados e responde conforme o prompt.

Resultados impressionantes

Alguns testes mostraram como a IA consegue:

Identificar pessoas em fotos e descrever elementos como roupas, objetos e contexto da cena.

Processar uma imagem de um documento e extrair campos como CNPJ, nome da empresa, endereço, telefone e status.

Trabalhar com diferentes parâmetros de **temperature** e **topK** para ajustar o nível de criatividade da resposta.

Por exemplo, uma imagem com um homem, um laptop e um cachorro foi descrita corretamente como um ambiente relaxante ao entardecer.

Tradução e compatibilidade com idiomas

Apesar de a API funcionar em português para prompts de texto, os recursos multimodais (imagem e áudio) ainda apresentavam melhores resultados em inglês. Para contornar isso, foi usada uma API de tradução embutida:

Input em português → traduzido para inglês.

Processado pela IA.

Output em inglês → traduzido de volta para português.

Esse processo de ida e volta é transparente ao usuário e ocorre rapidamente.

Casos de uso

Os exemplos apresentados demonstram potencial para aplicações como:

Acessibilidade, com leitura de imagens e transcrição de áudio.

Processamento de documentos offline.

Sites com assistentes visuais e auditivos.

Isso amplia o escopo de aplicação de IA no front-end, especialmente para dispositivos móveis.

Considerações finais

A capacidade de executar IA multimodal no navegador muda completamente a forma como pensamos sobre web apps. Agora, é possível criar assistentes que veem, ouvem e respondem, com desempenho aceitável e sem comprometer a privacidade.

Estamos no limiar de uma nova geração de experiências digitais, e você já pode experimentar essas tecnologias hoje mesmo. O futuro da IA na web é local, privado, multimodal e acessível.

Módulo 06: Prompt Engineering

Capítulo 1: Como escrever prompts que realmente geram boas respostas

Neste capítulo, vamos entender por que tantas pessoas ainda enfrentam dificuldades em extrair bons resultados de ferramentas de inteligência artificial como o ChatGPT. Seja por frustração com alucinações, respostas erradas ou pela necessidade de repetir a tarefa diversas vezes, o problema geralmente está na formulação dos prompts.

Aqui, trago uma introdução prática e direta sobre como melhorar seus prompts com base em conceitos fundamentais da engenharia de prompt, sem precisar mergulhar ainda nos aspectos mais avançados.

Por que a IA alucina?

A IA não pensa como um humano. Ela não é determinística; ela trabalha prevendo a próxima palavra mais provável com base no contexto que recebeu. Quando o prompt está mal formulado, o modelo tenta adivinhar o que você quer, o que abre espaço para erros. Surgem as chamadas "alucinações": o modelo inventa dados, mistura contextos ou responde com muita segurança mesmo sem ter informação suficiente.

Exemplo: ao perguntar "Quem é Eric Wendel?", a IA poderia misturar dados verdadeiros com falsas informações, como dizer que ele é autor de livros de Java, o que não corresponde à realidade.

Reduzindo Alucinações com Prompts Estruturados

Para evitar essas falhas, apresento aqui uma estrutura de prompt inspirada em um famoso estudo da Anthropic. Essa estrutura se baseia em 10 blocos que ajudam a guiar o modelo com clareza:

1. Contexto da tarefa: Defina o perfil da IA. Exemplo: "Você é Joe, um coach de carreira especializado em transição para a área de tecnologia".

2. Tom de voz: Estilo de comunicação. Formal? Empático? Didático? Isso influencia a forma como o modelo se expressa.
3. Fonte da verdade: Insira documentos, regras, tabelas ou qualquer material de referência que sirva como base factual para a resposta. Isso evita que a IA se baseie em dados genéricos da internet.
4. Contrato operacional: Regras de comportamento. Exemplo: "Se não tiver certeza, diga que não sabe. Se faltar dado, solicite informações." Isso cria um "protocolo" de segurança para a IA.
5. Exemplos: Mostre padrões de entrada e saída. Isso ancora a resposta esperada, define formato, estrutura e o que fazer quando faltar algum campo.
6. Histórico do usuário: Em aplicações mais robustas, esse campo traz o que já foi dito pelo usuário para manter consistência.
7. Pedido claro: Qual é a tarefa? Não confunda contexto com a demanda. Dê uma instrução objetiva.
8. Incentivo ao raciocínio: Peça para validar ou revisar antes de responder, se fizer sentido. Em tarefas complexas, pode ajudar a IA a refletir.
9. Formato da resposta: JSON, texto corrido, tabela? Especifique.
10. Restrições e validação: Limites de caracteres, idioma de resposta, obrigatoriedade de campos, e o que fazer quando um dado não estiver disponível.

Dicas práticas anti-alucinação

Um checklist que uso com frequência:

- Sempre inclua o papel da IA.
- Forneça documentos ou dados reais, mesmo que em texto simples.
- Peça para a IA não inventar, e sim dizer que não tem informação.
- Oriente o modelo a fazer perguntas quando o pedido for incompleto.
- Para ambiguidades, oriente a listar opções e pedir uma escolha.

Exemplos práticos

Se você pede: "Crie um plano de carreira pra mim", o modelo pode alucinar porque não sabe:

- Qual é sua área.
- Qual seu nível atual.
- Quais seus objetivos.

Um prompt melhor seria: "Você é um consultor de carreira. Meu objetivo é migrar para a área de desenvolvimento backend em Java. Tenho experiência com banco de dados e sou formado em engenharia. Crie um plano de carreira para três anos com foco em empresas de tecnologia."

Considerações finais

Prompts bem escritos economizam tempo, reduzem retrabalho e aumentam a precisão das respostas. Essa estrutura é uma base poderosa para qualquer pessoa que deseje usar IAs com mais eficiência, especialmente em contextos profissionais.

Nos próximos módulos, vamos aprofundar ainda mais em engenharia de prompt. Mas com o que você aprendeu aqui, já é possível evitar os erros mais comuns e tirar o máximo das IAs que você já usa no dia a dia.

Capítulo 2: Padrão TOON e JSON para Prompts

Neste capítulo, vamos entender como estruturar prompts de forma mais eficiente e como isso pode melhorar tanto a precisão quanto a economia de tokens. Exploramos dois formatos principais: o tradicional JSON Prompt e o novo formato conhecido como TOON (Token Oriented Object Notation).

LLMs gostam de dados estruturados

Apesar da impressão de que estamos conversando com um humano, LLMs são algoritmos treinados para prever tokens com base em entrada textual. Isso significa que, quanto mais estruturada for essa entrada, melhor o modelo consegue processar e responder.

O JSON Prompt surge como uma solução natural: é familiar, estruturado, previsível e integrável com ferramentas que desenvolvedores já usam, como Zod, Ajv e Yup.

Vantagens do JSON Prompt

Redução de ambiguidade: Campos separados evitam misturas de regras e instruções.

Previsibilidade de integração: Mais fácil validar saídas e gerar re-prompts automáticos.

Padronização para times e pipelines: Permite versionar, reutilizar e automatizar prompts com mais segurança.

Redução indireta de tokens: Embora JSON puro possa consumir mais tokens por causa da sintaxe, a padronização e prevenção de erros economizam em correções e retrabalho.

Estrutura sugerida para JSON Prompt

meta: nome, versão, idioma, papel da IA.

context: dados de referência ou informações base.

task: a tarefa desejada.

constraints: regras e limites.

output: formato esperado de resposta.

Essa separação ajuda a IA a entender com clareza o que deve ser feito e como.

TOON: a versão compacta para economia de tokens

TOON (Token Oriented Object Notation) surgiu com o objetivo de reduzir o custo em tokens, simplificando a estrutura sintática do JSON. Ele remove aspas, chaves e outros símbolos, deixando apenas a estrutura essencial.

Exemplo de ganho:

Lista de 6 itens em JSON: 367 tokens.

Lista de 6 itens em TOON: 339 tokens.

Com listas maiores, o ganho cresce exponencialmente, chegando a diferenças de mais de 140 tokens por prompt.

JSON bem estruturado ainda é competitivo

Mesmo com a proposta enxuta do TOON, um JSON bem modelado, especialmente quando representa dados tabulares, pode atingir eficiências próximas ou melhores. Um array de colunas e linhas elimina redundância de chaves e ainda continua sendo JSON válido, com toda a vantagem do ecossistema (validação, parsing, logging).

Exemplo:

JSON tabular: 26 tokens.

TOON equivalente: 35 tokens.

Quando usar cada um

JSON Prompt:

Quando precisa integrar com APIs.

Quando precisa validar saída com esquemas.

Quando é importante manter compatibilidade com ferramentas existentes.

TOON:

Quando a prioridade é máxima economia de tokens.

Quando a estrutura é simples e bem controlada.

Em pipelines onde o parsing customizado é aceitável.

Cuidados com TOON

Exige aprendizado de novo formato.

Não suportado por ferramentas comuns.

Pode trazer mais complexidade na integração.

Conclusão

O uso de JSON estruturado é hoje uma das melhores práticas para obter respostas mais precisas, previsíveis e com menos alucinações. O TOON vem como alternativa interessante em cenários muito específicos onde a economia de tokens é essencial.

Minha recomendação é clara: comece com JSON. Estruture bem, otimize seu prompt, use esquemas de validação. Isso já resolve grande parte dos problemas de integração com LLMs. TOON pode ser explorado mais adiante, quando houver maturidade na implementação.

No próximo capítulo, veremos como colocar tudo isso em prática para construir assistentes que realmente funcionam em ambiente de produção.

Módulo 07: Ferramentas de IA para acelerar sua vida como Dev

Capítulo 1: História e visão geral do Cursor, Comparando VSCode, Windsurf e Cursor

Estamos vivenciando uma verdadeira revolução na forma como interagimos com os editores de código. O que antes era apenas um ambiente para digitar comandos se transformou em um ecossistema de execução de agentes de inteligência artificial. Neste capítulo, vamos explorar a história e proposta do Cursor, e compará-lo com dois concorrentes de peso: o VS Code e o Windsurf.

O VS Code como base

Tanto o Cursor quanto o Windsurf são forks do VS Code. E isso não é coincidência. O VS Code se consolidou como um padrão da indústria por seu ecossistema maduro, extensões, temas, debugger integrado, terminal e suporte a Git. Ele virou o "Chrome" dos editores, e empresas viram a oportunidade de não reinventar a roda: aproveitar essa base sólida e adicionar camadas de IA.

Cursor: o fork AI First

O Cursor é um editor criado pela empresa Hemisphere em 2022, com uma abordagem totalmente voltada para IA desde o início. Não é apenas um VS Code com assistente embutido. Ele foi pensado para ser um ambiente de desenvolvimento assistido por IA, com foco em produtividade.

O investimento no Cursor foi massivo: US\$ 2,3 bilhões levantados em uma única rodada, com valor de mercado chegando a US\$ 29,3 bilhões. Essa dimensão chamou a atenção global, e é explicada pelo fato de que o Cursor e seus similares atacam diretamente o custo mais alto da indústria de software: o tempo de desenvolvimento.

Windsurf: outra aposta forte

O Windsurf também é um fork do VS Code, criado pela equipe do Codium, que ficou conhecida pelo plugin Kodi, uma alternativa ao autocomplete do GitHub Copilot. Ele teve uma proposta similar ao Cursor, integrando IA como protagonista do editor.

Seus números também impressionam: US\$ 150 milhões captados com valuação de US\$ 1,2 bilhão. A OpenAI chegou a considerar comprar o Windsurf por US\$ 3 bilhões, mas o acordo não foi concretizado. Mesmo assim, o projeto ganhou força e é hoje usado em larga escala.

Diferenças práticas no uso

Apesar das diferenças nas propostas, muitos desenvolvedores não percebem uma grande distinção entre o uso cotidiano do VS Code e do Windsurf. Os atalhos, extensões e a aparência geral são praticamente os mesmos.

O que muda é a profundidade da integração com IA. No VS Code, por exemplo, o recurso de chat com IA inclui três agentes nativos:

- ASCII: para perguntas rápidas e explicação de código.
- EDIT: para modificações controladas com base em guias.
- PLAN: para criação de planos de implementação antes da execução.

Após o plano aprovado, entra em ação o AGENT, que executa as tarefas com autonomia, iterando conforme o contexto e ferramentas disponíveis. Esse agente é muito utilizado por desenvolvedores em tarefas do dia a dia.

Agentes customizáveis

Uma das funcionalidades mais avançadas é a criação de agentes customizados no VS Code. A ideia é definir personas com tarefas repetitivas que fazem parte do ciclo de desenvolvimento. Esses agentes podem:

- Validar testes.
- Verificar boas práticas.
- Operar com modelos diferentes.
- Ter acesso restrito a ferramentas.

Na prática, eles funcionam como sub-prompts pré-definidos que executam tarefas específicas com escopo bem limitado.

Prompt vs. Editor

Ao longo do curso, o autor compartilhou uma experiência pessoal ao criar um aplicativo usando uma plataforma de Prompt-to-App (como Lovable). Apesar do início promissor, o app logo apresentou falhas graves por falta de segurança, validação e estrutura de código. Foi necessário migrar o projeto para o VS Code e reescrevê-lo com boas práticas.

Esse exemplo reforça a importância de usar editores como VS Code, Cursor ou Windsurf, que oferecem:

- Controle sobre o código.
- Revisão e testes.

- Auditoria.
- Integração com ferramentas de CI/CD.

Conclusão

Estamos diante de uma nova geração de editores, onde a IA deixa de ser um acessório e passa a ser o centro da experiência. O Cursor se destaca como um dos principais expoentes dessa tendência, mas tanto o Windsurf quanto o VS Code estão na mesma corrida para transformar o ambiente de desenvolvimento em um verdadeiro cockpit de produtividade assistida.

Para este curso, recomenda-se fortemente o uso de um editor com IA integrada. Independentemente da escolha entre VS Code, Cursor ou Windsurf, o que importa é manter o controle, aplicar boas práticas e explorar o potencial dos agentes inteligentes com responsabilidade.

Capítulo 2: O que são agentes de IA e como eles tomam decisões em etapas

Neste capítulo, vamos entender o que realmente são agentes de IA, como eles funcionam na prática e por que estão revolucionando a forma como desenvolvemos software. Diferente de apenas gerar respostas em linguagem natural, os agentes têm um ciclo de vida estruturado, composto por decisão, execução e validação.

A limitação das LLMs isoladas

Modelos de linguagem (LLMs) são excelentes em prever a próxima palavra ou token com base no contexto textual. No entanto, elas não sabem executar comandos, abrir arquivos, rodar testes ou validar seus próprios resultados. Por isso, surgiram os agentes de IA: sistemas que usam uma LLM como motor de decisão, acoplados a ferramentas, ciclos de execução e observação dos resultados.

O ciclo de um agente

Um agente transforma o prompt em uma tarefa concreta por meio de um ciclo composto por:

- Entendimento do objetivo.
- Planejamento da execução.
- Seleção de ferramentas.
- Ação.
- Observação do resultado.
- Correção de erros.
- Entrega final com evidências.

Esse ciclo garante que a tarefa não seja apenas imaginada, mas executada e validada. Assim, deixa-se de apenas conversar com a IA para realmente resolver um problema.

Agentes não são exclusivos de editores

Apesar de muitos agentes serem integrados a editores como VS Code, nada impede que estejam em qualquer lugar: back-ends, pipelines de CI, bots de suporte, sistemas de observabilidade, etc. O ambiente e as ferramentas é que mudam, mas o conceito é o mesmo.

A mentalidade de desenvolvedor

Para que um agente seja eficaz, ele precisa pensar como um desenvolvedor experiente:

- Planeja em etapas pequenas.
- Define critérios de aceitação.
- Cria plano de execução.
- Sabe quando algo está concluído.

Essa abordagem evita loops infinitos e desperdício de tokens, pois cada passo é validado antes de seguir para o próximo.

Escolha inteligente de ferramentas

O coração de um agente está na capacidade de escolher a ferramenta certa para cada tarefa:

- Ler código? Ferramenta de leitura de arquivos.
- Validar comportamento? Teste automatizado.
- Garantir estilo? Linter ou formatador.
- Buscar API? Integração com Swagger ou documentação.
- Ver banco de dados? Cliente de consulta SQL.

Esse raciocínio é o mesmo que desenvolvedores usam no dia a dia. O diferencial é a autonomia para executar isso.

Iteração com base em feedback

Um bom agente não apenas gera uma resposta. Ele:

1. Age.
2. Observa o que aconteceu.
3. Compara com a expectativa.
4. Corrige se for necessário.

Esse ciclo de feedback é o que traz a sensação de que a IA "acertou de primeira", pois ela teve a chance de ajustar antes de te mostrar o resultado final.

Agentes como papéis especializados

Agentes também podem ser divididos em papéis com responsabilidades específicas:

- Planner: cria o plano.
- Implementer: escreve e edita código.
- Reviewer: analisa os diffs e aponta riscos.
- QA: valida fluxo fim a fim.
- Docs Agent: escreve README e changelog.
- Ops Agent: monitora e sugere mitigação de incidentes.

Essa segmentação permite controle fino de permissões e reduz riscos.

Como evitar alucinação: Spec Driven Development

Agentes falham quando a tarefa está mal definida. Buracos no prompt são preenchidos com "chutes". Para resolver isso, usamos a técnica de Spec Driven Development:

Uma boa "spec" (especificação) deve incluir:

- Contexto: stack, ambiente, dependências.
- Requisitos: o que deve estar presente.
- Não-requisitos: o que deve ser evitado.
- Critérios de aceite: como saber se está pronto.
- Contrato: formato da API, shape da resposta.
- Plano de testes: como validar a funcionalidade.

Isso é fundamental para que o agente saiba quando parar e como validar o sucesso.

Um fluxo profissional com agentes

1. Defina a especificação (spec).
2. Crie os agentes com papéis definidos.
3. Execute cada etapa com validação.
4. Integre os resultados com testes automatizados.

O resultado é um fluxo de trabalho sem caos, com consistência e alta qualidade.

Conclusão

Agentes de IA não são apenas LLMs com plugins. São sistemas completos com controle de execução, planejamento e validação. Usar agentes não é mais opcional em projetos sérios: é uma forma de transformar sua produtividade e escalar com qualidade. E como sempre, o segredo está na engenharia de prompt. Ela é a base para construir agentes que funcionam de verdade.

Módulo 08: MCPs e Automação para Devs

Capítulo 1: O que são MCPs (Model Context Protocol) e como conectam IA com APIs/serviço

Neste capítulo, vamos explorar uma das inovações mais relevantes na integração entre inteligência artificial e sistemas reais: o Model Context Protocol, ou simplesmente MCP. Trata-se de um padrão criado para conectar modelos de linguagem com APIs, arquivos, bancos de dados e qualquer outra fonte de dados ou serviço.

O que é MCP?

O MCP foi anunciado pela Anthropic em novembro de 2024 como um protocolo aberto (open source) para integrar assistentes de IA com fontes de dados externas. A ideia central é que você possa simplesmente plugar servidores MCP prontos em um cliente compatível, como o VS Code, e a IA passará a utilizar essas integrações automaticamente, sem necessidade de codificação adicional.

Imagine pedir para a IA buscar um resumo das alterações solicitadas em um pull request, sugerir correções, aplicar os ajustes, escrever testes e comitar tudo automaticamente. Ou então, pedir que envie um e-mail diretamente pela API do Gmail. Tudo isso se torna viável com o uso de MCPs.

Como um MCP funciona

Um servidor MCP expõe três componentes fundamentais:

1. Tools (ferramentas): ações que a IA pode executar, como "listar palestras", "criar arquivo", "executar consulta SQL".
2. Resources (recursos): dados usados como contexto, como conteúdo de arquivos, logs ou esquemas de banco de dados.
3. Prompts: templates e estruturas que ajudam a IA a formular comandos adequados para usar essas ferramentas.

Esses elementos são descritos por schemas padronizados. A IA, ao interpretar o prompt do usuário, identifica qual tool melhor se encaixa com base no nome,

descrição e parâmetros esperados. É exatamente como um desenvolvedor humano: entende o problema e seleciona a função mais adequada.

Um exemplo prático: servidor MCP pessoal

O professor compartilhou a criação de um servidor MCP pessoal que expõe uma API com informações sobre palestras, posts e vídeos que ele publicou. Para isso, ele:

- Estruturou a API.
- Gerou um SDK automático via GraphQL.
- Criou testes automatizados com Node.js Test Runner.
- Publicou o pacote no npm.

Com isso, qualquer pessoa pode instalar essa integração e consultá-la diretamente da IA no editor, como o VS Code.

Benefícios do MCP

- Redução de alucinação: a IA acessa dados reais em vez de inventar informações.
- Extensibilidade: basta instalar novos servidores MCP para adicionar funcionalidades.
- Modularidade: você pode combinar múltiplas fontes em uma mesma interação.

Exemplo: perguntar quais foram as palestras do Eric em 2024, resumir a mais recente e enviar por e-mail.

Como as LLMs escolhem as ferramentas

As LLMs não contêm uma lógica de if-else para decidir qual ferramenta usar. Elas aprendem a selecionar a melhor opção com base na similaridade entre o prompt e a descrição da tool. Tools com nomes claros (ex: `readFile`) e descrições objetivas são priorizadas.

Além disso, a IA costuma preferir:

- Ações não destrutivas (como ler e listar) antes de escrever ou deletar.
- Tools com schemas de parâmetro bem definidos.

Modelos mais avançados conseguem executar sequências de chamadas: usar uma tool, analisar o resultado, e então chamar outra tool com base nessa resposta.

Casos de uso do mundo real

O professor mostrou exemplos de MCPs usados na prática, como:

- MCP do GitHub: listar PRs, revisar código, abrir PR.
- MCP do Playwright: criar testes e navegar em sites.
- MCP do Grafana: buscar dados de monitoramento.
- MCP de e-mail da Resend: gerar e enviar e-mails direto do editor.

Esses MCPs são plugados via arquivos de configuração ([mcp.json](#)) no VS Code ou outro editor compatível. Basta apontar para o servidor e fornecer as credenciais. A partir disso, a IA já está apta a usar essas integrações no seu fluxo de trabalho.

JSON Schema e validação

Cada tool define um JSON Schema para os parâmetros de entrada. A LLM precisa gerar um JSON válido para que a chamada seja executada. Se o schema não for atendido, a execução falha, e o modelo pode tentar novamente.

Isso traz um controle forte e previsibilidade para a interação entre IA e ferramentas, algo essencial para uso em ambientes de produção.

Conclusão

O MCP representa um salto na evolução da IA aplicada: sair do texto e entrar na ação concreta. Com ele, a IA não apenas responde, mas age com base em dados reais, seguindo padrões claros e seguros.

Esse tipo de integração deve se tornar cada vez mais comum, especialmente em ambientes corporativos onde segurança, auditabilidade e produtividade

caminham lado a lado. MCP é o "USB" das ferramentas para a era das LLMs: plug and play com poder real.

Capítulo 2: Usando IA para: Gerar testes automatizados

Neste capítulo, vamos demonstrar na prática como utilizar inteligência artificial, via MCP do Playwright, para gerar testes automatizados com qualidade e rapidez. O objetivo é claro: eliminar desculpas e tornar obrigatória a entrega de software com testes desde o início.

O contexto da demonstração

Toda a demo foi feita em um repositório vazio no VS Code. Utilizando apenas prompts estruturados e integração com agentes de IA, foi possível:

- Criar o projeto com o Playwright.
- Gerar um arquivo de teste base.
- Executar os testes.
- Ajustar falhas automaticamente.
- Integrar com GitHub Actions.

A ferramenta principal foi o servidor MCP do Playwright, que permite que a IA interaja com o navegador de forma autônoma e gere os scripts necessários.

Como a IA executa os testes

A IA usa o MCP para se comunicar com o navegador via comandos como:

- Abrir página.
- Preencher formulários.
- Submeter dados.
- Verificar resultados.
- Validar campos e erros.

Tudo isso é feito com base em prompts simples e objetivos, sem necessidade de codificação manual.

Etapas do fluxo automatizado

1. Inicialização do projeto: o prompt inicial define como a estrutura deve ser criada. O agente instala dependências, configura navegadores (ex: apenas Chromium) e valida se o ambiente está operacional com um teste inicial.
2. Criação de estrutura de CI/CD: outro prompt gera automaticamente o arquivo de workflow do GitHub Actions, integrando a execução dos testes com o repositório.
3. Mapeamento da página: a IA navega na página, identifica os elementos, campos de formulário, listas e botões.
4. Geração dos testes: baseado em prompts com objetivos genéricos (ex: validar submissão, verificar atualização da lista), a IA gera scripts que utilizam as funcionalidades do Playwright.
5. Execução e ajustes: os testes são rodados, e em caso de falhas, a IA ajusta automaticamente o código até que tudo passe.

Agente inteligente e interação com o browser

Durante a execução, a IA decide se deve abrir o Chrome em modo visível ou headless. Com base nos logs e nos resultados das interações, ela ajusta os comandos, modifica seletores e reescreve os testes conforme necessário.

Os relatórios visuais do Playwright ajudam a entender o que foi feito, e cada prompt executado pode ser reusado, refinado e testado com diferentes modelos, como GPT-5.

Vantagens da abordagem

- Produtividade: criação de testes com zero linha de código escrita manualmente.
- Padronização: estrutura inicial e CI gerados de forma uniforme.

- Validação real: os testes realmente rodam, são validados e documentados.
- Redução de erros: agentes ajustam automaticamente falhas comuns.
- Acesso facilitado: integração com editores e execução via comandos visuais.

Recomendação profissional

Para times de desenvolvimento modernos, essa abordagem representa uma mudança de paradigma. Ferramentas como o Playwright MCP integradas com agentes inteligentes tornam obsoleta a desculpa de que "não deu tempo de escrever teste".

A execução da IA mostra que com poucos prompts bem definidos é possível obter:

- Testes com cobertura significativa.
- Validação de casos críticos.
- Integração com pipeline de entrega.

Conclusão

Automatizar testes com IA é uma realidade acessível. A combinação entre prompt engineering, servidores MCP e agentes inteligentes abre caminho para um desenvolvimento mais robusto, validado e com foco em qualidade desde o início.

Se você ainda não utiliza IA para gerar testes, este é o momento ideal para começar. O impacto na produtividade, qualidade do software e tranquilidade na entrega é imediato e comprovado.

Capítulo 3: Usando IA para: Navegar em sites e extrair informações

Neste capítulo, demonstro como a inteligência artificial, por meio de ferramentas como o Playwright MCP, está revolucionando a forma como interagimos com páginas web. Com alguns prompts simples, é possível automatizar completamente tarefas como preenchimento de formulários, extração de informações e navegação entre seções de sites, sem escrever uma linha de código.

Cenário prático: o problema dos formulários repetitivos

Quem participa com frequência de eventos sabe o quão desgastante é preencher várias vezes os mesmos dados pessoais em formulários diferentes. Esse foi meu ponto de partida. Utilizei o Playwright MCP dentro do VS Code para automatizar esse processo com base nas informações do meu perfil em uma plataforma de eventos que reúne minhas palestras em diversos idiomas.

Passo a passo da automação com Playwright MCP

1. Configuração do MCP: No VS Code, configurei o agente de IA apontando para o MCP do Playwright. Esse agente é capaz de navegar em páginas reais, clicar em elementos, preencher campos e muito mais. O JSON de configuração é colado no painel do agente e adaptado conforme as extensões instaladas no navegador.
2. Conexão com o navegador: Com a extensão Playwright MCP Bridge instalada no Chrome, foi possível permitir que a IA interagisse diretamente com as abas abertas. Isso é essencial para cenários que exigem autenticação ou sessão logada.
3. Execução dos comandos: A partir de um prompt como "Navegue no site ericwendel.com e me resuma o que existe aí", a IA se conectou ao navegador, abriu a página, percorreu as seções e começou a coletar dados.
4. Preenchimento automatizado de formulário: Com o contexto da página obtido, o agente acessou um formulário real da comunidade Node.br e iniciou o preenchimento automático com dados retirados do site do Sessionize, onde mantenho meu perfil como palestrante.

5. Gerenciamento de contexto e continuidade: A IA manteve o contexto em memória, mesmo após múltiplas interações. Por exemplo, quando faltaram dados como telefone ou e-mail, o agente solicitou diretamente essa informação, ajustou o que precisava e continuou preenchendo.
6. Alternando palestras e refinando prompts: Foi possível trocar a palestra automaticamente, limpar o formulário e repetir o processo com outro conteúdo. Essa interatividade permite personalizar os resultados com prompts refinados e mais objetivos.

Benefícios observados

- Eficiência: tarefas demoradas e repetitivas são eliminadas.
- Precisão: a IA extrai dados diretamente da fonte, reduzindo erros manuais.
- Escalabilidade: o mesmo fluxo pode ser reaproveitado para dezenas de formulários.
- Integração local: é possível rodar modelos de IA localmente com ferramentas como o Olyama, economizando custos com tokens.

Execução local e open source

Uma parte avançada da aula mostrou como rodar o mesmo tipo de automação usando modelos locais. Usei o modelo QuenCoder de 30 bilhões de parâmetros via a plataforma Tome, que integra modelos open source com servidores MCP. Mesmo com poder computacional limitado, foi possível executar tarefas de navegação e extração com qualidade satisfatória.

Considerações finais

Este tipo de integração é um divisor de águas. A IA, combinada com servidores MCP e extensões no navegador, é capaz de agir como um verdadeiro agente digital, navegando, preenchendo, interagindo e aprendendo com o ambiente online.

Para desenvolvedores web, profissionais de dados ou qualquer pessoa que lide com tarefas repetitivas em páginas web, essa é uma revolução silenciosa, mas extremamente poderosa.

Com o aprimoramento constante dos modelos e da engenharia de prompts, fica cada vez mais claro que automatizar processos não é mais uma opção distante: é uma realidade presente, acessível e altamente produtiva.

Capítulo 4: Usando IA para: Consultar documentações atualizadas

Um dos maiores desafios no uso de LLMs no dia a dia é lidar com informações desatualizadas. Modelos de linguagem como GPT ou Claude são treinados até uma certa data, e a partir dali não aprendem mais, a menos que a gente forneça dados recentes. Isso se torna um problema quando APIs, bibliotecas ou frameworks mudam de versão, e o modelo segue sugerindo códigos antigos, quebrados ou incompatíveis.

Foi justamente para resolver esse problema que surgiu o Context-7, um servidor MCP criado com o objetivo de fornecer documentação atualizada, precisa e diretamente da fonte para o modelo de linguagem.

O que é o Context-7

O Context-7 é um servidor que indexa documentações completas de projetos reais, como Next.js, BetterOff, Node.js, Prisma, entre outros. Ele consegue puxar os trechos mais relevantes da documentação no momento da execução do prompt e inserir automaticamente no contexto que será processado pela IA. Com isso, o modelo passa a gerar códigos e respostas baseadas em versões atuais das ferramentas.

Diferença no fluxo de desenvolvimento

Sem o Context-7, a prática comum é colar blocos enormes da documentação manualmente no prompt, mandar links e reexplicar o setup a cada tentativa. Isso

consome tokens, torna o processo caro e ainda propenso a erros. Com o Context-7, o prompt fica muito mais leve, porque o servidor MCP faz a seleção dos trechos precisos e enxutos automaticamente.

Exemplo prático: criando uma aplicação com Next.js e BetterOff

Na demonstração, o objetivo foi criar uma aplicação fullstack usando Next.js como framework e BetterOff para autenticação com GitHub. A persistência de dados foi feita com SQLite, facilitando o ambiente local.

Para isso, o prompt usado foi bem estruturado:

- Definiu que o agente é um desenvolvedor fullstack experiente.
- Deixou claro que o Context-7 deve ser usado obrigatoriamente.
- Descreveu passo a passo o que a aplicação precisa conter.
- Pediu para criar o projeto na pasta **PromptMD**.

Integração do Context-7 no VS Code

O processo de integração foi simples:

1. Criação da API Key no painel do Context-7.
2. Configuração do arquivo **mcp.json** com a chave.
3. Habilitação das tools **queryDocs** e **resolveLibrary** no agente.
4. Execução do prompt diretamente no VS Code.

A partir disso, o agente começou a interagir com o Context-7, consultando a documentação do Next.js, do BetterOff e de outras ferramentas necessárias, sempre trazendo apenas o essencial para a tarefa.

Benefícios claros

- Precisão: o código gerado está sempre alinhado com a versão atual da ferramenta.
- Menor custo: prompts menores, menos tokens, menos tentativas.
- Segurança: reduz drasticamente erros por uso de métodos obsoletos.
- Produtividade: o agente consegue criar uma aplicação funcional do zero com um único prompt.

Execução final da aplicação

Ao final da execução:

- O projeto foi criado com `npm init` usando Next.js.
- Dependências foram instaladas automaticamente.
- O banco local SQLite foi inicializado e migrado.
- O fluxo de login com GitHub foi configurado.
- O projeto rodou localmente na porta 3000.

O agente utilizou o Context-7 para buscar documentação e exemplos de integração com autenticação, banco de dados e configurações do Next.js. Todo o processo foi guiado por esse prompt inicial, e a IA foi capaz de completar as tarefas sem necessidade de correções manuais.

Considerações finais

O Context-7 representa um novo patamar no uso de IA para desenvolvimento. Ele garante que os modelos estejam alinhados com o estado atual do mundo, evitando erros comuns de alucinação e desatualização. Isso torna o uso de LLMs mais confiável, mais eficiente e muito mais poderoso.

Se você ainda não usa um servidor MCP como o Context-7 nos seus projetos, este é o momento ideal para começar.

Capítulo 5: Usando IA para: Colher dados de Telemetria de apps

Neste capítulo, exploro como colocar a inteligência artificial para atuar como uma verdadeira detetive digital, utilizando dados de telemetria para investigar falhas em aplicações. O foco é usar AI para interpretar traces, logs e métricas, correlacionando tudo isso em tempo real para produzir relatórios automatizados e precisos.

A dor de quem trabalha com bugs intermitentes

Quem já lidou com sistemas em produção sabe: bugs que aparecem e somem são um pesadelo. Logs isolados nem sempre ajudam, e reproduzir o erro pode ser inviável. É aqui que a telemetria entra como solução: ela permite registrar tudo que está acontecendo na aplicação por meio de métricas, logs e traces, entregando uma visão detalhada do comportamento do sistema.

A infraestrutura da aula

Utilizei uma aplicação chamada Aluminus, já preparada para emitir dados de telemetria. A instrumentação foi feita com OpenTelemetry, que é hoje o padrão mais adotado para coleta unificada de telemetria.

Esses dados foram enviados para:

- Prometheus: coleta e armazena métricas de performance.
- Grafana Tempo: armazena os traces.
- Grafana Loki: centraliza os logs.
- Grafana: visualiza e correlaciona os sinais em painéis interativos.

Tudo foi orquestrado com Docker Compose para simular um ambiente real de produção.

O papel do Grafana MCP e da IA

Para transformar esses dados em conhecimento acionável, utilizei um servidor MCP conectado ao Grafana. Com isso, um agente de IA integrado ao VS Code foi capaz de fazer investigações reais com base em um único prompt: "Estou recebendo erro 500 neste endpoint, descubra o motivo e gere um relatório."

Como funciona a investigação automatizada

1. Coleta de métricas: a IA acessa o Prometheus e verifica, por exemplo, que todos os requests estão retornando erro 500.
2. Exploração de logs: acessa o Grafana Loki para analisar os logs relacionados, identificando mensagens de erro.
3. Tracing: consulta o Grafana Tempo para reconstruir a cadeia de chamadas e localizar gargalos.

4. Correlação dos sinais: cruza os dados das três fontes e descobre que o problema está relacionado a um vazamento de conexões com o banco de dados.

O mais incrível é que a IA executa toda essa análise sem acesso ao código-fonte. Ela atua apenas com base nos dados expostos pelas ferramentas de telemetria.

Diagnóstico e relatório final

O agente gerou um relatório estruturado com:

- Nome do endpoint afetado.
- Tempo de resposta das requisições.
- Stack trace com falha na conexão com o banco.
- Causa raiz: múltiplas conexões sendo criadas a cada requisição, sem reutilização.
- Linhas suspeitas de código onde o leak ocorre (mesmo sem ter acessado o repositório).

Impacto e aplicação real

Essa prática foi usada para resolver um problema real de vulnerabilidade na minha infraestrutura em produção. O poder de usar IA com telemetria permitiu encontrar e explicar rapidamente falhas graves de performance.

Por que usar essa abordagem?

- Redução do tempo de investigação: uma tarefa que levaria horas pode ser feita em minutos.
- Precisão no diagnóstico: evita achismos e foca nos dados concretos.
- Menor dependência do código-fonte: ideal para sistemas legados ou desconhecidos.
- Alta integração com ferramentas do mercado: funciona com tecnologias amplamente utilizadas.

Considerações finais

Colocar a IA para trabalhar com telemetria é o futuro da observabilidade. Ferramentas gratuitas e poderosas estão à nossa disposição, como Grafana, OpenTelemetry e Prometheus. Integradas a agentes inteligentes via MCP, transformam dados brutos em diagnósticos claros, rápidos e confiáveis.

Se você ainda está usando logs manuais e debugging tradicional, considere experimentar essa abordagem. O ganho de produtividade e clareza é imediato e vale cada segundo investido na configuração inicial.

Módulo 09: Modelos open-source vs. proprietários

Capítulo 1: Vantagens e desvantagens de modelos abertos e fechados

Neste capítulo, exploro com profundidade as diferenças entre modelos de linguagem abertos e fechados, com base na minha experiência ao buscar alternativas menos restritivas e mais controláveis para o uso de IA no dia a dia. A reflexão parte de uma situação concreta: uma simples dúvida sobre como funciona um emulador de videogame levou a uma cadeia de descobertas sobre os limites dos modelos populares e a abertura para o universo dos modelos open source.

Modelos Abertos: o que realmente significa?

No mundo dos LLMs, "aberto" raramente significa o mesmo que em projetos open source tradicionais. Em geral, quando falamos de modelos abertos estamos nos referindo a modelos que permitem o download e execução local ("open weights"), mas não necessariamente com acesso ao pipeline de treinamento ou à base de dados utilizada.

Sites como o Olyama.com são bons exemplos disso. Com comandos simples, como `olhama pull` e `olhama serve`, você pode baixar modelos como o LLaMA 3.0 da Meta, o Gemma do Google ou o GPT OSS da OpenAI e executá-los

localmente, inclusive offline. Alguns desses modelos estão marcados como "sem censura", permitindo explorações mais livres.

Vantagens dos Modelos Abertos

- **Custo Reduzido:** para quem já possui infraestrutura local (como um PC gamer com boas GPUs), é possível prototipar sem custos de API.
- **Privacidade e Controle:** ao rodar localmente, você elimina a necessidade de enviar dados sensíveis a terceiros. Isso é essencial para quem lida com informação confidencial.
- **Customização:** é possível ajustar o modelo ao seu caso de uso, aplicar fine-tuning, criar variantes especializadas e integrar em produtos sem limitações externas.
- **Independência de Fornecedor:** você não depende das decisões de negócio de empresas como OpenAI ou Google, que podem mudar regras, limites ou preços de uma hora para outra.

Desvantagens dos Modelos Abertos

- **Infraestrutura Custosa:** manter um ambiente local com suporte a modelos grandes é caro. Requer placas de vídeo potentes, refrigeração, energia, distribuição e monitoramento.
- **Manutenção Complexa:** é preciso cuidar de atualizações, compatibilidades, segurança e escalabilidade. Isso demanda tempo e conhecimento.
- **Limitações Legais:** modelos como o LLaMA possuem licenças restritivas, limitando uso comercial em certos contextos. A legalidade do uso pode variar conforme o porte da sua empresa.
- **Falta de Filtros:** modelos "sem censura" podem gerar respostas perigosas, imprecisas ou inapropriadas. Isso exige governança e cuidado redobrado, especialmente em aplicações voltadas a clientes finais.

Modelos Fechados: por que ainda fazem sentido?

Apesar dos benefícios dos modelos abertos, os modelos fechados, como os da OpenAI, Google e Anthropic, ainda são a melhor escolha para muitos cenários empresariais. Eles oferecem:

- Qualidade superior: os modelos de ponta em benchmarks ainda são, em geral, fechados.
- Facilidade de uso: basta uma API key para começar. Toda infraestrutura e atualização ficam por conta do fornecedor.
- Escalabilidade garantida: é possível atender milhões de usuários sem se preocupar com distribuição de carga.
- Suporte e estabilidade: contratos comerciais garantem SLA, segurança e conformidade legal.

Hoje, muitos desenvolvedores optam por hubs de IA que permitem trocar dinamicamente de modelo conforme o uso e o perfil de tarefa, pagando apenas por aquilo que realmente utilizam.

Reflexão Final

Se você está explorando IA para uso pessoal, testes locais ou prototipagem, modelos abertos oferecem um universo de possibilidades, desde que você tenha estrutura e conhecimento para mantê-los. Já para ambientes empresariais, com necessidades de escalabilidade, conformidade e confiabilidade, os modelos fechados ainda têm grande vantagem.

O mais importante é entender que nenhum dos dois caminhos é universalmente melhor. O segredo está em conhecer os limites e benefícios de cada um e escolher o modelo mais alinhado ao seu contexto técnico, financeiro e legal.

Capítulo 2: Como rodar modelos localmente com Ollama

Neste capítulo, compartilho minha experiência pessoal com o Ollama, uma ferramenta extremamente acessível e poderosa para baixar, gerenciar e

executar modelos de linguagem de forma local, sem necessidade de conexão com servidores externos. Ao longo do capítulo, explico o funcionamento da ferramenta, suas possibilidades, desafios e a prática de utilização com foco em programadores interessados em IA.

O que é o Ollama?

O Ollama é uma aplicação que funciona em MacOS, Windows e Linux, com uma interface simples e uma API HTTP por trás, permitindo integrações com qualquer aplicativo. Criado em cima do ecossistema da Meta e dos modelos LLaMA, o Ollama ganhou destaque a partir de julho de 2023. Seu grande diferencial está na facilidade de uso: com comandos simples como `ollama pull` e `ollama serve`, é possível baixar e executar modelos localmente.

O Ollama possui um amplo catálogo de modelos, incluindo modelos gerais, de código e modelos multimodais, que permitem entradas combinadas de texto e imagem.

Benefícios do Ollama

- Execução local e offline: ideal para automações, scripts, testes e estudos.
- Gratuito: não exige pagamento por tokens.
- Compatibilidade com vários modelos: LLaMA, GPT OSS, Gemma, entre outros.
- Interface amigável e terminal integrável: permite integração com curl, scripts shell, editores e servidores MCP.

Limitações

Apesar das vantagens, o Ollama não é indicado para uso em produção. Ele executa apenas um prompt por vez, demanda muitos recursos da máquina local e não oferece alta concorrência. Em ambientes com necessidades de escalabilidade e baixa latência, ferramentas como VLLM são mais apropriadas.

Parâmetros, Contexto e Quantização

Para entender o comportamento dos modelos, explico três conceitos importantes:

- **Parâmetros:** representam os pesos do modelo, ou seja, o que ele aprendeu. Mais parâmetros geralmente indicam maior capacidade, mas também mais exigência computacional. Exemplos: modelos 20B (bilhões de parâmetros) e 120B.
- **Tamanho de contexto:** define quantos tokens (palavras, frases, comandos) podem ser processados por vez. Por exemplo, 32k, 128k ou até 1M tokens.
- **Quantização:** processo de reduzir o tamanho do modelo, substituindo a representação numérica dos pesos. Modelos quantizados usam menos memória e rodam mais rápido, com uma leve perda de qualidade.

Sufixos como **Q4F32__1** indicam os esquemas de quantização aplicados (ex.: pesos em 4 bits, ativações em 32 bits). Isso permite rodar modelos de 20B com apenas 16GB de RAM.

Experimentação com Curl e Interface HTTP

Mostro como é simples criar scripts que utilizam o Ollama via terminal. Com a API HTTP, podemos fazer requisições **curl** para enviar prompts e receber respostas em JSON. Isso permite integração com qualquer sistema, inclusive para automação local.

A execução pode usar endpoints compatíveis com a API da OpenAI, o que facilita portar aplicações existentes para o Ollama.

Modelos com ou sem censura

Exploro a diferença entre modelos censurados e modelos "sem censura". Enquanto alguns modelos limitam respostas que envolvem temas sensíveis, outros não impõem filtros. Isso pode ser útil em ambientes de pesquisa ou estudo, mas exige responsabilidade, especialmente em aplicações de uso geral.

Integração com Jan AI

Apresento o Jan AI, uma aplicação desktop open source que permite usar modelos locais com funcionalidades semelhantes ao ChatGPT, incluindo assistentes, histórico, banco vetorial, file system e integração com MCPs.

O Jan AI suporta modelos de várias fontes (OpenRouter, Hugging Face, Ollama, etc.) e permite configurar o MCP localmente, o que viabiliza interação com arquivos do sistema, navegação web, e uso em projetos pessoais.

Jan AI no Editor de Código

Demonstro como integrar o Jan AI no editor de código para usar modelos locais em atividades como geração de código, refatoramento e revisão. Basta adicionar o provedor Ollama e selecionar o modelo desejado. Assim, mesmo modelos pequenos podem ajudar no dia a dia sem qualquer custo.

Considerações Finais

Rodar modelos localmente com o Ollama é uma excelente alternativa para programadores que desejam explorar IA de forma econômica e independente. Apesar das limitações em ambientes de produção, para prototipagem, automação local e estudo, a experiência é incrível.

Ao dominar ferramentas como Ollama e Jan AI, você potencializa suas aplicações de IA, reduz custos e ganha liberdade para experimentar. O segredo está em compreender as limitações de hardware, escolher modelos adequados ao seu cenário e aplicar boas práticas de prompt engineering.

Capítulo 3: Como usar OpenRouter para orquestrar vários modelos

Neste capítulo, compartilho como utilizo o OpenRouter para orquestrar diferentes modelos de linguagem em um único ponto de integração. A motivação para isso veio da dificuldade de manter várias integrações simultâneas com APIs de

diferentes fornecedores, cada uma com seu SDK, formatos, limitações e custos distintos.

Por que usar um orquestrador?

Quando você começa a usar diferentes modelos (como OpenAI, Google, Anthropic, etc.), percebe que o trabalho de integração vira um caos: diversas API keys, SDKs, regras, dashboards de billing espalhados e um risco real de vendor lock-in. Com um orquestrador como o OpenRouter, tudo isso é centralizado.

OpenRouter é uma API unificada, compatível com o padrão da OpenAI, que permite trocar de modelo ou provedor por configuração, sem alterar a lógica da aplicação. Isso muda completamente o jogo para aplicações reais, oferecendo flexibilidade, escalabilidade e manutenção simplificada.

Funcionalidades do OpenRouter

- Fallback automático: se um modelo falha, ele tenta outro automaticamente.
- Roteamento por custo ou desempenho: você define suas prioridades (como latência ou preço) e ele escolhe.
- Cobrança consolidada: um único billing, mesmo que você use modelos de diferentes provedores.
- Modelos gratuitos: muitos modelos não exigem cartão de crédito.
- Compatibilidade com padrões: APIs, formatos e autenticação semelhantes ao da OpenAI.

Como Começar

O processo é simples:

1. Acesse o site do OpenRouter.
2. Crie uma conta gratuita.
3. Navegue pelos modelos disponíveis, aplicando filtros como "100% Free".
4. Gere sua API Key e configure variáveis de ambiente como OPENROUTER_KEY e outras.

Com a chave gerada, você pode começar a testar modelos gratuitos com controle total.

Casos de uso reais e modelos

Durante a aula, usei o modelo Gemma 27B, que possui 27 bilhões de parâmetros e um desempenho interessante para tarefas gerais de linguagem. Mesmo sendo gratuito, ele entregou respostas surpreendentes.

Outros modelos também estão disponíveis, como o LLaMA 2, Mistral e modelos menores como 4B ou 2B que podem ser utilizados até no navegador.

Você pode classificar modelos por tipo (texto, imagem, embeddings) e por custo. A maioria dos modelos multimodais e de embeddings ainda não possui opções gratuitas, mas isso pode mudar com o tempo.

Cuidados com a segurança

Durante a demonstração, destaquei a importância de manter as chaves de API fora do controle de versão. Um erro comum é subir o arquivo de configuração para o GitHub e expor a chave. O OpenRouter tem mecanismos de segurança para invalidar chaves vazadas automaticamente, mas o ideal é sempre proteger esses dados.

Integrações Bring Your Own Key (BYOK)

Além dos modelos já disponibilizados, é possível plugar suas próprias chaves da OpenAI, Anthropic, entre outros. Isso permite manter o controle sobre sua cota enquanto aproveita os benefícios do orquestrador.

Conclusão

Usar o OpenRouter é uma excelente estratégia para quem deseja liberdade na escolha de modelos, economia de custos e simplicidade na integração com aplicações reais. Para protótipos, testes e até mesmo alguns usos em produção, ele se mostra uma solução robusta e acessível.

Recomendo fortemente que você experimente o OpenRouter nos seus projetos, explore diferentes modelos e construa sua própria arquitetura de IA sem amarras.

Módulo 10: RAG, embeddings e busca semântica

Capítulo 1: O que é RAG (Retrieval-Augmented Generation) e por que ele é tão importante

Neste capítulo, explico em detalhes o conceito de RAG (Retrieval-Augmented Generation), por que ele é essencial na arquitetura de sistemas com LLMs, como ele funciona e de que forma se diferencia de outras abordagens como fine-tuning e MCP.

A Essência do RAG

O RAG é uma arquitetura que adiciona um passo fundamental antes da geração de resposta por uma LLM: a busca de informação externa relevante. Ou seja, em vez do modelo depender apenas do que foi aprendido durante o seu treinamento, ele passa a contar com informações injetadas dinamicamente no contexto.

Isso permite que a LLM gere respostas com base em dados atualizados, privados ou específicos de um domínio, algo que seria inalcançável apenas com os dados embutidos em seus parâmetros.

Memória Paramétrica vs. Não Paramétrica

RAG se apoia em dois tipos de memória:

Memória paramétrica: é o conhecimento aprendido durante o treinamento e codificado nos pesos do modelo.

Memória não paramétrica: é uma base de dados externa, pesquisável, que geralmente utiliza um índice vetorial para identificar trechos relevantes para cada pergunta.

O conceito de RAG surgiu formalmente em 2020 com um paper que consolidou essa ideia de combinar os dois tipos de memória para melhorar a precisão das respostas.

O Problema da Ausência de Contexto

Para entender o valor do RAG, precisamos lembrar como um transformer funciona: ele prediz o próximo token com base no contexto atual. Se o contexto fornecido não contém a informação correta, o modelo tende a completar a resposta com base em padrões aprendidos, o que pode gerar erros ou alucinações.

Com o RAG, em vez de tentar confiar na memória do modelo, garantimos que o contexto contenha os trechos certos na hora certa. Isso muda o jogo.

Como Funciona na Prática

A aplicação do RAG se divide em duas fases:

Indexação:

Coletamos fontes de dados como documentos, tickets, códigos, tabelas, PDFs.

Esses dados são divididos em pedaços coerentes (chunks).

Cada pedaço recebe um embedding (vetor numérico) e é armazenado em um banco vetorial.

Consulta:

O usuário envia uma pergunta.

A pergunta também é convertida em embedding.

O sistema busca os embeddings mais similares no banco vetorial.

Os trechos mais relevantes são injetados no prompt para a LLM.

Assim, a resposta gerada está fundamentada em informação real e contextualizada.

Casos Reais de Uso

Imagine um chatbot corporativo que responde a perguntas sobre procedimentos internos, postmortems, diagnósticos ou código legado. Com RAG, o sistema busca e injeta apenas os trechos relevantes desses documentos, sem necessidade de treinar novamente o modelo.

Exemplo clássico: o usuário pergunta "Por que o endpoint /checkout está dando erro 500?". O sistema, usando embeddings, busca trechos de runbooks, incidentes e código e injeta isso no prompt, possibilitando uma resposta fundamentada, com base em evidências.

Diferença entre RAG, MCP e Fine-Tuning

RAG: é o método de buscar informação relevante antes de responder. Ele melhora a qualidade sem alterar os pesos do modelo.

MCP (Model Context Protocol): é um protocolo de integração que permite que o modelo acesse ferramentas externas, como buscadores, editores, sistemas de arquivos. Pode ser usado como canal para o RAG, mas não é um método de recuperação em si.

Fine-tuning: é o processo de ajustar os pesos do modelo com novos exemplos. A memória se torna permanente, mas exige custo, tempo e conhecimento para treinar.

Dar contexto com RAG é diferente de treinar. O modelo apenas usa as informações enquanto estão no prompt. Ao final da interação, esse conhecimento é descartado, diferente do que ocorre com fine-tuning.

Considerações Finais

RAG é uma das formas mais poderosas de tornar LLMs realmente úteis em cenários corporativos. Permite acesso a conhecimento privado, atualizações constantes e evita a alucinação por falta de contexto. A chave está no uso eficiente de embeddings e em um banco vetorial bem estruturado.

Nos próximos capítulos, vamos explorar na prática como esses embeddings funcionam e como você pode construir seu primeiro sistema RAG do zero. Se você quer aumentar a precisão, reduzir erros e integrar conhecimento real no seu sistema de IA, dominar RAG é essencial.

Capítulo 2: O que são Embeddings e Vector Databases

Quando falamos de inteligência artificial hoje em dia, a primeira imagem que surge na mente da maioria das pessoas é a de um chatbot respondendo perguntas de forma articulada. No entanto, por trás dessa interface interativa e sofisticada, existe um componente muito mais fundamental e menos glamoroso: os embeddings e a busca por similaridade. Esses são os verdadeiros motores que tornam as soluções de IA realmente úteis e relevantes para os usuários.

Neste capítulo, quero mostrar de maneira prática como isso funciona. A proposta é apresentar a mecânica que sustenta o funcionamento de sistemas de IA, sem truques ou aparências polidas, e sem a interferência de modelos generativos. Vamos explorar o conceito de embeddings, como eles são gerados, armazenados e utilizados em buscas por similaridade dentro de um banco de dados vetorial, utilizando o Neo4j.

Entendendo Busca por Similaridade

Diferente do comando Ctrl+F usado para encontrar palavras exatas em documentos, a busca por similaridade trabalha com o sentido e não com a forma literal das palavras. Ao transformar trechos de texto em vetores (os embeddings), conseguimos identificar similaridades sem depender de palavras idênticas. Dois

textos com significados parecidos, mesmo que escritos com palavras distintas, serão representados por vetores próximos no espaço vetorial.

Imagine que você quer saber o que é backpropagation, ou como uma rede neural ajusta os pesos, ou ainda como funciona um gradiente descendente. Mesmo que esses termos específicos não estejam na transcrição de uma aula, um sistema baseado em embeddings é capaz de encontrar trechos relevantes onde esses conceitos são explicados com outras palavras.

Embeddings são representações numéricas que codificam o significado de um texto. Podemos imaginá-los como um mapa em que ideias semelhantes estão geograficamente próximas. Quando fazemos uma consulta, o sistema apenas faz os cálculos matemáticos para determinar quais vetores estão mais próximos do vetor da consulta.

O Papel do Neo4j como Banco Vetorial

Apesar de ser conhecido como um banco de dados de grafos, o Neo4j também funciona excelentemente como um vector database. Nele, armazenamos os pedaços de texto (chunks) como nós do grafo, e os embeddings como propriedades desses nós. Criamos um índice vetorial que permite consultas por similaridade, retornando os trechos mais relevantes de acordo com a proximidade dos vetores.

Uma das vantagens desse tipo de abordagem é a possibilidade de enriquecer os dados. Cada chunk pode ter metadados associados, como o título da aula, a seção, o minuto do vídeo e outros. Isso possibilita consultas mais estáveis e resultados com maior contextualização.

Geração Local de Embeddings

Para essa experiência, utilizei a biblioteca Transformer.js para gerar os embeddings localmente. Isso significa que conseguimos fazer tudo sem depender de chaves de API, sem usar Docker e sem sobrecarregar a máquina. Tudo funciona em Node.js puro.

O fluxo é simples: pegamos uma transcrição de aula em PDF, quebramos em pedaços menores (chunks), geramos embeddings desses pedaços e armazenamos no Neo4j. Em seguida, é possível realizar perguntas e visualizar os trechos mais semelhantes com base apenas na similaridade vetorial. É o puro funcionamento do mecanismo de RAG (Retrieval-Augmented Generation), mas sem a parte de "generation" ainda.

Da Teoria à Prática

Durante a prática, usei um exemplo com uma transcrição de aula sobre tensores. Peguei o arquivo PDF e, através de uma pipeline local, transformei o texto em chunks e gerei os embeddings para cada um. Cada trecho foi indexado no Neo4j com metadados. Em seguida, executei consultas como "o que é one hot encoding?" ou "como treinar uma rede neural?". Mesmo sem essas frases exatas na transcrição, o sistema foi capaz de retornar parágrafos altamente relevantes.

Esse processo deixa claro que a busca por similaridade é um componente essencial para a IA moderna. Antes mesmo de um modelo generativo responder algo, ele precisa de um contexto relevante, e esse contexto é fornecido por um mecanismo como esse.

Conclusão

A compreensão profunda de embeddings e vector databases é vital para qualquer desenvolvedor que deseje trabalhar com sistemas de IA mais robustos e confiáveis. Eles são a espinha dorsal das aplicações baseadas em RAG e são o elo entre o armazenamento eficiente e a inteligência contextual das respostas. No próximo capítulo, vamos conectar esse mecanismo de embeddings com um modelo de linguagem para realizar respostas completas, fechando o ciclo do RAG com geração de linguagem natural.

Capítulo 3: Projeto: Criando o primeiro RAG com JavaScript e Neo4j

Neste capítulo, compartilho como criei meu primeiro projeto completo de RAG utilizando JavaScript e Neo4j. A proposta foi transformar uma transcrição de aula em uma base consultável, capaz de responder perguntas com base em busca por similaridade, antes mesmo de usar modelos generativos.

Comecei o projeto com a leitura e preparação do conteúdo em formato PDF. Dividi a transcrição em pedaços coerentes, chamados de *chunks*, cada um representando um trecho com sentido completo. Em seguida, utilizei a biblioteca [transformer.js](#) para gerar os embeddings localmente. Isso significa que não dependi de serviços externos, chaves de API ou infraestrutura pesada — tudo rodando localmente, de forma leve e eficiente.

Esses embeddings, que são vetores numéricos representando o significado de cada chunk, foram armazenados em um banco vetorial dentro do Neo4j. Cada chunk ficou associado a metadados como o minuto da aula, o nome do arquivo e a posição no texto. Isso trouxe não apenas contexto, mas também a possibilidade de organizar e refinar as buscas posteriormente.

Na segunda fase do projeto, implementei uma etapa de consulta por similaridade. O sistema converte as perguntas em embeddings, compara com os armazenados no Neo4j e retorna os trechos mais próximos. Essa etapa permitiu criar um sistema de memória não paramétrica, fundamental para o funcionamento do RAG, garantindo que o modelo receba sempre o contexto certo antes de gerar qualquer resposta.

A construção dessa base deixou claro o quanto a engenharia de dados é essencial para IA. O modelo só é bom quando alimentado com dados relevantes e bem estruturados. Ter controle sobre os embeddings, sobre os metadados e sobre a forma como os trechos são indexados me deu autonomia para criar uma arquitetura de IA verdadeiramente útil.

Finalizo este capítulo com a certeza de que dominar esse tipo de projeto é o passo mais importante para qualquer desenvolvedor que queira levar IA a sério. Com esse tipo de estrutura, podemos alimentar modelos com conhecimento real, atualizado e alinhado ao nosso domínio. E o melhor: sem depender de soluções prontas ou fechadas.

